Posterita Unified Platform — Master Plan v3.7

Single canonical document. Supersedes all prior versions.

Version	Date	Key Changes
v1.0	19 Mar 2026	Initial plan — 25 sections, architecture, sync model
v2.0	19 Mar 2026	Full data model, API catalog, implementation sequence
v3.0	19 Mar 2026	Merged with addendum: loyalty schema, inventory count dual-scan, WhatsApp, reports, brand portal scrape
v3.1	19 Mar 2026	All 14 concerns resolved (C1–C14), AI-assisted catalogue enrichment, QR acquisition funnel, spot check, Zebra printer, configurable points
v3.2	**19 Mar 2026**	Receipt QR mandatory on every transaction
v3.3	**19 Mar 2026**	Owner signup, multi-brand hierarchy, AI product generation during onboarding
v3.4	**19 Mar 2026**	In-app chat + AI assistant, Logistics & delivery, Universal QR deep routing, Operational supplies
v3.5	**19 Mar 2026**	Cash on delivery, QR label printer mandatory per store
v3.6	**19 Mar 2026**	Android Dynamic Feature Modules, Store Cash Collection, "Barcode My Store" guided workflow, AI product ID from photos
v3.7	**19 Mar 2026**	**Container/Import receiving at warehouse — document vault, inspection workflow, flexible release-to-store, purchase order with landed cost allocation. New role: Merchandiser.**

---

1. Executive Summary

Build one unified Posterita Retail OS comprising:

- **One Android shell app** for all internal store operations (POS, staff ops, supervisor, inventory count, loyalty)
- **One web control console** for device management, approvals, reconciliation, reports, campaigns, and operations oversight
- **One backend platform** serving both surfaces plus AI agent integration and WhatsApp customer messaging
- **One WhatsApp integration** for customer-facing loyalty, consent capture, and digital receipts

The current `Posterita Brand Portal` (loyalty.posterita.com) provides the operational model for loyalty, shifts, campaigns, attendance, and consent. The loyalty Flask API provides the POS integration contract (award, balance, consent, voucher validate/redeem). Both are absorbed into the unified platform. An early-stage Android POS codebase exists as a playground — it will be evaluated for reusable patterns and code, but architecture and long-term goals take precedence over preserving playground code. Zoho Creator is retired; Supabase Postgres is the sole source of truth.

Design Principles

1. **Offline-first** — every store-floor operation must work without connectivity and reconcile cleanly when connectivity returns.
2. **Capability-driven** — what a user sees and can do is a function of their role, their device assignment, and the store context. No hardcoded screen lists.
3. **Auditable by default** — every mutation of business consequence produces an audit event. No silent writes.
4. **AI-operable** — the system exposes CLI, API, MCP, and queue-based interfaces so agents can participate in operations without touching device UI.
5. **Architecture-first** — the existing playground codebase is evaluated for reuse, but long-term architecture always wins over preserving short-term work.
6. **Scan-only inventory** — inventory count is performed entirely by scanning. No manual data entry. Dual-device verification on every full count shelf.
7. **Catalogue-as-product** — the POS product selection screen doubles as the source for printed catalogue PDFs, barcode labels, and showroom materials.
8. **QR-first customer acquisition** — every QR code in the physical environment (on products, shelves, catalogues, receipts, storefronts) funnels into WhatsApp, which captures the customer's phone number and starts the loyalty onboarding pipeline. The QR code is the primary customer acquisition mechanism for the entire Retail OS.

MVP Scope Boundary

In scope: Retail POS, Staff Ops, Supervisor, Multi-device Inventory Count with dual-scan verification, Loyalty (wallets, points, vouchers, campaigns, consent), Reconciliation, Device/Capability management, Shift planning with attendance QR, WhatsApp customer messaging with Flows, Reports (daily sales, discrepancies, device health, count results, enrollment), Shelf label printing, CRM read-only sync for support, AI-assisted product catalogue + onboarding, In-app AI assistant chat, Basic logistics (package labeling + standard delivery), Operational supplies (non-resale inventory), Universal QR deep routing.

Phase 3 additions: Direct + group chat, full delivery templates (motorcycle handover, inter-store transfer), driver assignment, container/import receiving with document vault + inspection + claims + cost allocation, "Barcode My Store" guided workflow, purchase orders.

Explicitly deferred: Restaurant suite, queue management, bookings, order-ahead, marketing email campaigns, support/social inbox, Deliveroo integration, full HR engine, payroll deductions, advanced costing/freight/duty, utility bills, full AI copilot, customer-facing loyalty web portal, Paddle billing, driver route optimization, supplier management, procurement.

---

2. Current State and Migration Context

What Exists Today

System	Role Today	Fate in Unified Platform
Posterita Android POS (playground)	Early-stage POS codebase with initial work done. Not production — no live stores, no historical transaction data to preserve.	Evaluate for reusable code (layouts, Blink integration patterns, scanner handling). Adopt good patterns into the new architecture. Rebuild what doesn't fit. Architecture > preservation.
Posterita Brand Portal (Next.js on Vercel)	Brand operations console: 13 sections (products, catalogue PDF, points, staff, shifts, campaigns, audience/consent, vouchers, operations health, billing), connected to 21 Zoho Creator reports	Absorbed into unified web console; Vercel portal instance retired
Zoho Creator	Source of truth for 21 operational reports: products, wallets, ledger, consent, staff, shifts, attendance, campaigns, vouchers, stores, holidays, admin log	All data modeled fresh in Supabase. Creator retired as data store.
Loyalty Flask API (Vercel)	Proxy layer between brand portal and Zoho Creator; blueprints for balance, award, consent, voucher	Retired. Business logic moves to NestJS backend modules.
Zobot WhatsApp scripts (SalesIQ)	1,228-line customer-facing bot: enrollment, consent, points, name update, catalogue, agent transfer. Two variants (Posterita loyalty + Yadea showroom).	Bot logic moves to backend. SalesIQ stays as WhatsApp middleware. Zobot becomes a thin 50-line relay.
Zoho CRM	Contains test contact data, custom modules (Loyalty_Points, Loyalty_Transactions), and consent fields. NOT live production data — was a playground.	**Not a migration source.** Becomes one-way sync target: backend pushes customer/loyalty status so SalesIQ support team can see context.
Xero	Accounting system of record	No change; reconciliation exports built as needed
Cloudinary	Product images, catalogue assets	Retained; same role in unified platform

⚠ CONCERN: What the portal prototype reveals

The brand portal at loyalty.posterita.com is more sophisticated than initially described — it has 13 working sections with live Creator data. However, it's a **prototype/playground**, not production. Key evidence:

- Only 9 products (all Yadea electric vehicles, not Funky Fish retail)
- Only 2 loyalty wallets with small test balances (320 pts total)
- 1 staff record, 1 admin number
- Campaigns are drafts (queue tables "still need to go live")
- Billing is sandbox (Paddle keys missing)
- Voucher table is empty
- Audience shows only 4 consent rows, all for the same phone number

This means: We don't need to migrate data from Creator/CRM. We build the Supabase schema from scratch, informed by the portal's data model but not constrained by its test data. Fresh start.

Migration Approach (simplified from v2)

Since there's no production loyalty/shift/campaign data in Creator or CRM:

1. **No data migration needed** from Zoho Creator or CRM — build fresh in Supabase
2. **Android app migration** follows the incremental refactor approach (wrap existing POS, build shell around it)
3. **Reference data seeding** — products, stores, staff, Mauritius public holidays entered fresh or imported via CSV
4. **Dual-write is unnecessary** — no legacy system to keep in sync during transition

Remaining Migration Constraints

- **Stores are live.** The current Android app is running in production. Every refactoring step must keep the current app shippable.
- **Staff must not retrain twice.** The new app should feel familiar to cashiers. UX changes should be evolutionary.

---

3. Final Stack and Responsibilities

Layer	Technology	Responsibility
Android	Kotlin, Gradle, multi-module, Play App Bundle	Store-floor operations, offline-first, capability-driven UI
Web Console	Next.js on Vercel	Device/capability management, approvals, reconciliation, reports, campaigns, operations oversight
Backend API	NestJS modular monolith on Render	Business APIs, sync orchestration, loyalty engine, reconciliation, WhatsApp logic, CRM push, reports
Background Workers	NestJS workers on Render	Push ingest, loyalty jobs, WhatsApp dispatch, file processing, notifications, campaign delivery
Cron	Render cron service	Scheduled reconciliation, heartbeat stale-check, report generation, voucher expiry, consent renewal
Database	Supabase Postgres	Sole operational source of truth
Auth	Supabase Auth + backend-managed device/app sessions	Phone OTP, JWT, device enrollment tokens, session revocation
Realtime	Supabase Realtime	Dashboard live updates, device command inbox, approval status push, inventory count live dashboard
Cache/Queue	Redis (Render-hosted or Upstash)	Job queues (BullMQ), rate limiting, session cache, idempotency key store
Media	Cloudinary	Product images, reconciliation evidence, signatures, documents, catalogue assets
WhatsApp	Meta Cloud API via SalesIQ middleware	Outbound customer comms, loyalty notifications, WhatsApp Flows for registration
CRM Sync	NestJS `crm-connector` module	One-way push to Zoho CRM on customer/loyalty events for support visibility
Source Control	GitHub	Monorepo or multi-repo, PR-based workflow
Android CI/CD	GitHub Actions	Lint, test, build, sign, artifact publish
Android Cloud Testing	Firebase Test Lab	Instrumentation tests on real devices/emulators
Android UI Automation	Maestro	Business-flow end-to-end tests
Monitoring	Sentry (errors), Render metrics, Supabase dashboard, UptimeRobot	See §16
Receipt/Label Printing	Epson ePOS SDK for Android	Bluetooth, WiFi, USB Epson printers — receipts and shelf labels
Card Payments	Blink SDK (existing integration preserved)	Card/tap payments at POS, requires connectivity

---

4. High-Level Architecture

```
┌──────────────────────────────────────────────────────────────────────────┐
│                              CONTROL PLANE                               │
│  Users · Devices · Capabilities · Stores · Terminals · Shelves           │
│  Approvals · Audit · Agent Tasks · Shift Planning                        │
└──────────────────────────────────────────────────────────────────────────┘
              │                                  │
              │                                  │
    ┌─────────▼──────────┐          ┌──────────▼──────────────┐
    │   ANDROID SHELL    │          │   WEB CONSOLE           │
    │   (Store Floor)    │          │   (Back Office)         │
    │                    │          │                         │
    │  Base Runtime      │          │  Device Allocation      │
    │  Capability Loader │          │  Capability Mgmt        │
    │  Offline-First DB  │          │  Approval Dashboards    │
    │  Sync Engine       │          │  Reconciliation Review  │
    │  POS + Blink       │          │  Inventory Count Dashboard │
    │  Inventory Count   │          │  Reports & Analytics    │
    │  Loyalty at POS    │          │  Campaigns & Vouchers   │
    │  Staff Ops         │          │  Loyalty & Consent Mgmt │
    │  Shelf Label Print │          │  Shift Planning         │
    └────────────────────┘          │  Audit Trail            │
              │                     │  AI Task Center         │
              │                     └─────────────────────────┘
              │                                  │
    ┌─────────▼──────────────────────────────────▼──────────────────────┐
    │                  BACKEND PLATFORM (Render)                        │
    │                                                                   │
    │  API Service        Worker Service         Cron Service           │
    │  ────────           ──────────────         ────────────           │
    │  Business APIs      Sync ingest            Stale checks           │
    │  Auth/sessions      Loyalty jobs           Report gen             │
    │  Sync orch.         Campaign delivery      Voucher expiry         │
    │  Loyalty engine     File processing        Consent renewal        │
    │  WhatsApp logic     CRM sync push          Wallet reconcile       │
    │  Inv. count logic   Notifications                                 │
    │  Agent-control      Shelf match compute                           │
    │  Reports engine     Label PDF gen                                 │
    └───────────────────────────────────────────────────────────────────┘
              │                    │                    │
              │                    │                    │
    ┌─────────▼──────────┐  ┌──────▼──────┐  ┌──────────▼──────────┐
    │  Supabase Postgres │  │   Redis     │  │  Cloudinary         │
    │  (source of truth) │  │ (queue/cache)│  │  (media)            │
    │  Auth · RLS · RT   │  └─────────────┘  └─────────────────────┘
    └────────────────────┘
              │
              │ one-way push
    ┌─────────▼──────────┐        ┌──────────────────────────┐
    │  Zoho CRM          │        │  WhatsApp (via SalesIQ)  │
    │  (read-only mirror) │        │  Meta Cloud API          │
    │  Support visibility │        │  WhatsApp Flows          │
    └────────────────────┘        └──────────────────────────┘
```

Key Boundaries

- **Android never talks to Supabase directly** — all data flows through the backend API.
- **Web console mutations go through the backend API** — Supabase Realtime is for live updates only.
- **AI agents use CLI/MCP/API/queues** — never Android UI automation.
- **One store per user per day** — JWT carries `store_id` claim. No store switching.
- **WhatsApp bot logic lives in the backend** — SalesIQ is middleware only; Zobot is a thin relay.
- **Zoho CRM is write-only from our perspective** — backend pushes; never reads from CRM.
- **Inventory count is scan-only** — no manual data entry during counting.

---

5. Offline-First Architecture — One-Way Sync

Unchanged from v2. See v2 §5 for full detail on: - Sync direction registry (device-owned vs server-owned entities) - Server-side ingest pipeline (authenticate → dedup → detect version → transform → validate → import) - Outbox queue design - Transformer pattern - Idempotency handling - Room database strategy (destructive migration with `onCreate` callback)

Addition to sync direction registry:

Entity	Direction	Write Owner
InventoryShelfCount	Device → Server	Device
InventoryShelfCountLine	Device → Server	Device
Shelf	Server → Device	Server
Customer	Server → Device	Server (created via API, cached on device)
LoyaltyWallet	Server → Device	Server (read-only cache for POS lookup)
ShiftTemplate	Server → Device	Server
ShiftSelection	Device → Server	Device
AttendanceLog	Device → Server	Device

6. Platform Bootstrapping & Authentication

Entity Hierarchy

```
Owner (person)
  └── Account (billing entity)
        └── Brand (1..n)
              └── Store (1..n)
                    └── Terminal (1..n)
                          └── Device (1..1 per terminal)
```

An owner can operate multiple brands (e.g., Funky Fish retail + Yadea electric vehicles). Each brand has its own product catalogue, loyalty program, customer base, and branding. Stores belong to a brand. Terminals belong to a store. Devices are enrolled to terminals.

Flow A: Owner Signup (the "Chapter 0" — from zero to operational)

This is the front door to the entire Retail OS. It works on **both Android and web**. The flow is designed for minimum friction — no business registration number required, AI does the heavy lifting.

Android UX rule: The keyboard must NEVER hide the input field. Split every form into single-field steps. Capture one piece of information at a time with large, clear inputs and prominent "Next" buttons.

```
STEP 1: PHONE NUMBER
┌──────────────────────────────┐
│                              │
│   Welcome to Posterita       │
│                              │
│   What's your mobile number? │
│                              │
│   +230  [5XXX XXXX          ]│
│                              │
│   We'll send a code via      │
│   WhatsApp to verify         │
│                              │
│          [ Next → ]          │
│                              │
└──────────────────────────────┘

STEP 2: OTP VERIFICATION
┌──────────────────────────────┐
│                              │
│   Enter the code             │
│   sent to +230 5823 1102     │
│                              │
│   [ 5 ] [ 8 ] [ 3 ] [   ] ...│
│                              │
│   Resend via WhatsApp        │
│                              │
└──────────────────────────────┘

STEP 3: YOUR NAME
┌──────────────────────────────┐
│                              │
│   What's your name?          │
│                              │
│   [Fred                     ]│
│                              │
│          [ Next → ]          │
│                              │
└──────────────────────────────┘

STEP 4: BRAND NAME (AI starts here)
┌──────────────────────────────┐
│                              │
│   What's your brand called?  │
│                              │
│   [Funky Fish               ]│
│                              │
│   This is what customers see │
│                              │
│          [ Next → ]          │
│                              │
└──────────────────────────────┘

STEP 5: LOCATION
┌──────────────────────────────┐
│                              │
│   Where's your first store?  │
│                              │
│   [Grand Baie, Mauritius    ]│
│                              │
│   We'll set up your store    │
│   address and currency       │
│                              │
│          [ Next → ]          │
│                              │
└──────────────────────────────┘

STEP 6: PRODUCT CATEGORY
┌──────────────────────────────┐
│                              │
│   What do you sell?          │
│                              │
│   ○ Fashion & Apparel        │
│   ○ Footwear                 │
│   ○ Electronics              │
│   ○ Food & Beverage          │
│   ○ Health & Beauty          │
│   ○ Sports & Outdoor         │
│   ○ Home & Living            │
│   ○ Other: [            ]    │
│                              │
│          [ Next → ]          │
│                              │
└──────────────────────────────┘

STEP 7: AI BUILDS YOUR CATALOGUE
┌──────────────────────────────┐
│                              │
│   ✨ Setting up Funky Fish   │
│                              │
│   Creating your store...  ✓  │
│   Building products...    ⟳  │
│                              │
│   AI is generating a starter │
│   catalogue based on your    │
│   brand and category.        │
│                              │
│   You'll review each product │
│   before it goes live.       │
│                              │
└──────────────────────────────┘
```

What happens at Step 7 (backend):

1. Create `owner` record (phone, name)
2. Create `account` record (linked to owner)
3. Create `organization` record (brand name)
4. Create `store` record (location, inferred currency from country)
5. Create `organization_loyalty_config` with defaults for this category
6. **Trigger AI product generation job:** - Input to Claude API: brand name + location + product category - AI generates 15-30 starter products appropriate for this category and market - Each product: name, suggested price (in local currency), category, short description - Products are created as `product_enrichment` suggestions (NOT in the master `product` table) - Status: `ai_generated`, awaiting owner review

Flow B: Owner Reviews AI-Generated Products

Immediately after Step 7, the owner is presented with AI-generated products one by one:

```
PRODUCT REVIEW (1 of 24)
┌──────────────────────────────┐
│                              │
│   Funky Fish — Starter Product │
│                              │
│   ┌────────────────────────┐ │
│   │   🤖 AI Suggestion     │ │
│   │                        │ │
│   │   Beach Sandal Classic │ │
│   │   Rs 890               │ │
│   │   Category: Footwear   │ │
│   │                        │ │
│   │   "Essential open-toe  │ │
│   │   sandal for the       │ │
│   │   tropical lifestyle." │ │
│   └────────────────────────┘ │
│                              │
│   [ ✓ Accept ] [ ✎ Edit ]    │
│   [ ✗ Skip  ] [ ✗ Skip All ] │
│                              │
│   ━━━━━━━●━━━━ 1/24          │
│                              │
└──────────────────────────────┘

IF EDIT:
┌──────────────────────────────┐
│                              │
│   Edit product               │
│                              │
│   Name:                      │
│   [Reef Pro Sandal Navy     ] │
│                              │
│   Price:                     │
│   [1,290                    ] │
│                              │
│   Category:                  │
│   [Footwear              ▾  ] │
│                              │
│   Description:               │
│   [Durable reef sandal...   ] │
│                              │
│          [ Save → ]          │
│                              │
└──────────────────────────────┘
```

Rules: - **Accept** → product moves to master `product` table, `catalogue_ready = true` - **Edit** → owner modifies fields → then product moves to master table - **Skip** → product is discarded (not added to master) - **Skip All** → skip remaining AI suggestions, go to dashboard - Owner can always add more products later from the POS product grid or trigger AI enrichment again

Flow C: Owner Creates PIN & Completes Setup

After product review (or Skip All):

```
STEP 8: CREATE PIN
┌──────────────────────────────┐
│                              │
│   Create your login PIN      │
│                              │
│   ● ● ○ ○                    │
│                              │
│   [1] [2] [3]                │
│   [4] [5] [6]                │
│   [7] [8] [9]                │
│       [0] [⌫]                │
│                              │
└──────────────────────────────┘

STEP 9: SETUP COMPLETE
┌──────────────────────────────┐
│                              │
│   🎉 Funky Fish is ready!    │
│                              │
│   ✓ Account created          │
│   ✓ Brand: Funky Fish        │
│   ✓ Store: Grand Baie        │
│   ✓ 18 products approved     │
│   ✓ Loyalty program active   │
│                              │
│   You're the owner and admin.│
│   Invite staff from Settings.│
│                              │
│   [ Go to Dashboard → ]      │
│                              │
└──────────────────────────────┘
```

Flow D: Owner Login (returning)

Owner opens the app → recognized by device token:

```
┌──────────────────────────────┐
│                              │
│   Welcome back, Fred         │
│   Funky Fish                 │
│                              │
│   Enter your PIN             │
│                              │
│   ● ● ○ ○                    │
│                              │
│   [1] [2] [3]                │
│   [4] [5] [6]                │
│   [7] [8] [9]                │
│       [0] [⌫]                │
│                              │
│   Use biometric ▸            │
│                              │
└──────────────────────────────┘
```

Flow E: Device Login (staff — not owner)

Staff member opens app on an enrolled device:

```
┌──────────────────────────────┐
│                              │
│   Grand Baie · POS-GB-01     │
│   Funky Fish                 │
│                              │
│   Who's logging in?          │
│                              │
│   ┌────────────────────┐     │
│   │ 👤 Sarah M. — Cashier │   │
│   │ 👤 Ravi P. — Cashier  │   │
│   │ 👤 Amina K. — Supervisor│ │
│   └────────────────────┘     │
│                              │
│   Enter PIN                  │
│   ○ ○ ○ ○                    │
│                              │
└──────────────────────────────┘
```

The device knows which store it belongs to (from enrollment). It shows only staff assigned to that store. Staff selects their name → enters PIN → JWT issued with `user_id`, `store_id`, `role`, `capabilities`.

Roles & Permissions

Capability	Owner	Admin	Merchandiser	Supervisor	Driver	Cashier
Create brands/stores	✓					
Invite admins	✓					
Invite staff	✓	✓				
Manage products	✓	✓	✓			
Run AI enrichment	✓	✓	✓			

Approve AI suggestions	✓	✓	✓			
Container receiving & inspection	✓	✓	✓			
Manage purchase orders	✓	✓	✓			
Upload container documents	✓	✓	✓			
Release stock to stores	✓	✓	✓			
Configure loyalty	✓	✓				
View all stores	✓	✓	✓			
Approve leave/expense	✓	✓		✓		
Resolve discrepancies	✓	✓		✓		
Logistics / deliveries	✓	✓		✓	✓	
Cash collection transport	✓				✓	
POS operations	✓	✓		✓		✓
Inventory count	✓	✓	✓	✓		✓
Barcode My Store	✓	✓	✓			
View own data	✓	✓	✓	✓	✓	✓

Merchandiser is the person who manages the product pipeline: receiving containers, inspecting goods, processing import documents, maintaining product data quality, managing cost prices, and releasing stock to stores. They are the gatekeeper between the warehouse and the retail floor.

Account Schema

```
-- ═══════════ OWNER ═══════════
CREATE TABLE owner (
    id              UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    phone           TEXT NOT NULL UNIQUE,
    name            TEXT NOT NULL,
    email           TEXT,
    created_at      TIMESTAMPTZ NOT NULL DEFAULT now(),
    updated_at      TIMESTAMPTZ NOT NULL DEFAULT now()
);

-- ═══════════ ACCOUNT (billing entity) ═══════════
CREATE TABLE account (
    id              UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    owner_id        UUID NOT NULL REFERENCES owner(id),
    plan            TEXT NOT NULL DEFAULT 'free'
                    CHECK (plan IN ('free', 'starter', 'pro', 'enterprise')),
    billing_status  TEXT NOT NULL DEFAULT 'active'
                    CHECK (billing_status IN ('active', 'trial', 'past_due', 'cancelled')),
    trial_ends_at   TIMESTAMPTZ,
    created_at      TIMESTAMPTZ NOT NULL DEFAULT now(),
    updated_at      TIMESTAMPTZ NOT NULL DEFAULT now()
);

-- Organization already exists in the schema; add account reference:
ALTER TABLE organization ADD COLUMN account_id UUID REFERENCES account(id);
ALTER TABLE organization ADD COLUMN owner_id UUID REFERENCES owner(id);
```

AI Product Generation During Signup

The AI generates starter products using the same Anthropic API pipeline as the catalogue enrichment (§29), but with a different prompt:

```
You are setting up a new retail store.

Brand: {brand_name}
Location: {location}
Category: {product_category}
Currency: {currency}

Generate 20-30 starter products that a {product_category} store
in {location} would typically carry. For each product provide:
- name (specific, not generic — e.g. "Classic Reef Sandal Navy" not "Sandal")
- suggested retail price in {currency} (realistic for {location} market)
- sub-category
- short description (1 sentence)

Return JSON array. Be realistic about local pricing and product mix.
Do NOT invent brand names — use generic descriptive names.
```

Important: These products go into `product_enrichment` table (same as catalogue enrichment), NOT directly into the `product` table. The owner must accept/edit/skip each one. The same review UI and safety controls from §29 apply.

Cost: ~$0.02 per signup (one Claude API call generating 20-30 products). Negligible.

Token Lifecycle

Token	Lifetime	Storage	Revocation
Owner access JWT	15 minutes	Android Keystore / browser httpOnly cookie	Short-lived, natural expiry
Owner refresh JWT	30 days	Android Keystore / secure cookie	Server-side revocation
Staff access JWT	15 minutes	Android Keystore	Short-lived, natural expiry
Staff refresh JWT	7 days	Android Keystore	Server-side revocation list in Redis
Device Token	Until revoked	Android Keystore	`RevocationVersion` broadcast
Enrollment Token	24 hours, single-use	Backend only	Consumed on use

Owner gets longer refresh tokens (30 days) because they're the account holder and log in less frequently than daily staff.

Staff Onboarding (unchanged from v2 but now in context)

After the owner is set up:

1. Owner invites staff by phone via web console or Android Settings → `POST /v1/auth/invite-by-phone`
2. Staff receives WhatsApp OTP
3. Staff enters OTP on enrolled device → `POST /v1/auth/verify-otp`
4. Staff completes profile + creates PIN
5. Staff can now log in on any enrolled device in their store

Device Enrollment (unchanged from v2)

1. Owner/admin creates enrollment QR in web console or Android Settings
2. QR contains: `{ enrollment_token, store_id, capability_profile_id }`
3. Android app scans QR → `POST /v1/devices/enroll`
4. Backend validates, registers device, returns device session
5. Device downloads capability snapshot + initial data sync

Single-Store Binding

A user works in exactly one store per day. The store context is established at device enrollment. The JWT includes `store_id` as a claim. **Exception:** owners can switch between their stores/brands from a brand switcher in the app.

Device Revocation

Unchanged from v2: Revoke via web console → increment `RevocationVersion` → device receives 403 on next call → token wipe → data wipe → lock screen → uninstall prompt.

---

7. Android App Strategy

Module Structure — Android Dynamic Feature Modules

The app uses **Android Play Feature Delivery** to comply with Play Store policies and minimize install size. Only core modules install at first. Feature modules download on demand when the user first taps them.

```
INSTALL-TIME (always present, every device):
──────────────────────────────────────────────

:app                          ← Shell, DI root, QR deep router
:core:designsystem            ← Theme, components, brand tokens
:core:auth                    ← Login, PIN, biometric, token management
:core:device                  ← Enrollment, heartbeat, revocation
:core:data                    ← Room DB, DAOs, sync engine, offline queue
:core:navigation              ← Shared nav graph, deep links, QR payload router
:core:sync                    ← Outbox push, server pull, retry engine
:core:network                 ← API client, interceptors, connectivity observer
:core:printer                 ← Epson ePOS + Zebra ZPL abstraction
:core:scanner                 ← Barcode/QR scan: camera + Bluetooth HID
:core:media                   ← Cloudinary upload, camera, signature capture
:feature:home                 ← Role-first home screen, brand switcher (owner)

ON-DEMAND (downloaded when user first opens feature):
──────────────────────────────────────────────────────

:feature:onboarding           ← Owner signup, AI product review       [trigger: first launch, no account]
:feature:pos                  ← POS, cart, checkout, payment           [trigger: tap POS tile]
:feature:reconciliation       ← Close till, evidence, discrepancy      [trigger: tap close till]
:feature:inventory-count      ← Dual-scan count, spot check            [trigger: tap Inventory tile]
:feature:barcode-my-store     ← Guided first-time barcoding (§34)      [trigger: tap "Barcode My Store"]
:feature:warehouse            ← Container receiving, inspection, claims  [trigger: tap Warehouse tile]
:feature:loyalty              ← Earn/redeem/consent at POS             [trigger: first loyalty interaction]
:feature:catalogue            ← POS grid → PDF gen + barcode labels    [trigger: tap Catalogue tile]
:feature:chat                 ← AI assistant + messaging               [trigger: tap Chat tab]
:feature:logistics            ← Driver delivery, packages, COD, cash   [trigger: tap Logistics tile]
:feature:staff-ops            ← Attendance, leave, tasks, expenses     [trigger: tap Staff Ops tile]
:feature:supervisor           ← Approvals, checklists, shift mgmt      [trigger: tap Supervisor tile]
```

Play Feature Delivery implementation:

```
// In :core:navigation — when user taps a feature tile on home screen
fun navigateToFeature(feature: DynamicFeature) {
    val manager = SplitInstallManagerFactory.create(context)
    if (manager.installedModules.contains(feature.moduleName)) {
        navController.navigate(feature.startDestination)  // instant
    } else {
        showDownloadProgress(feature) // "Downloading POS module... 78%"
        val request = SplitInstallRequest.newBuilder()
            .addModule(feature.moduleName)
            .build()
        manager.startInstall(request)
            .addOnSuccessListener { navController.navigate(feature.startDestination) }
            .addOnFailureListener { showDownloadError(it) }
    }
}
```

Why this matters: - **Play Store compliant** — no unnecessary code downloaded - **Fast first install** — base app ~8MB. A cashier never downloads `:feature:logistics` or `:feature:supervisor` - **Owner testing** — owner downloads base → onboarding downloads (~3MB) → tries POS (~4MB) → tries inventory (~2MB). Each feature downloads in seconds on 4G. - **Home screen only shows tiles for features the user's role permits.** But even among permitted features, code isn't on disk until first tap.

Approach to Existing Playground Code

The existing Android POS codebase is a playground with initial work. The approach:

1. **Evaluate** the existing code for reusable patterns (layouts, navigation, Blink integration, scanner handling)
2. **Adopt** good code directly into the new module structure where it fits
3. **Rebuild** anything that doesn't align with the target architecture (offline-first, capability-driven, clean module boundaries)
4. **Architecture wins** over code preservation — don't compromise the long-term structure to save playground work
5. **No `:feature:pos-legacy` wrapper** — since this isn't a live production app, there's no need for the careful incremental wrapping approach. Build `:feature:pos` properly from the start.

`:core:scanner` — Configurable Scanning (C2 resolved)

Both camera and external Bluetooth barcode scanners are supported. The scanner module provides:

- **Camera-based scanning** via ML Kit or ZXing — default for devices without external scanners
- **Bluetooth HID scanner input** — external scanners that act as keyboard input, intercepted at the activity level
- **QR code scanning** — shelf labels, attendance QR stations, customer registration QR
- **Configurable modes:**
- **Auto-scan:** scan detected → immediately process → ready for next (fastest for inventory counting)
- **Scan-confirm:** scan detected → show product/shelf → user taps confirm → process (safest for POS)
- **Continuous batch:** rapid fire scanning with audio beep per scan, product list accumulates (for shelf counting)
- **Scan-to-resolution:** scanner emits a generic `ScanResult(barcode_data, format)` event. The consuming feature module resolves it (POS resolves to product, inventory resolves to shelf or product, attendance resolves to QR station).

`:core:printer` — Multi-printer Support (C7 resolved)

Printer Type	SDK	Use Case
Epson ePOS (receipt)	Epson ePOS SDK	POS receipts, small shelf labels
Zebra (label)	Zebra Link-OS SDK or ZPL direct	Enterprise shelf labels, product barcode labels
Generic (personal)	Android Print Framework	PDF catalogue pages, reports

The module exposes a `PrinterService` interface. Printer type is configured per device during enrollment. The consuming code never references a specific SDK.

`:feature:catalogue` — POS Screen Reuse for Catalogue (C3 resolved)

The POS product selection screen is reused for catalogue generation. Same product grid, same categories, same images — but with a different action mode:

- **POS mode:** tap product → add to cart
- **Catalogue mode:** select products → choose output format:
- **Print barcode labels** — generate barcode sheet for selected products (Zebra or Epson)
- **Generate PDF catalogue** — credit card/A5/flyer templates with product images, prices, WhatsApp QR codes (ported from brand portal catalogue studio)
- **Print shelf labels** — for selected shelf locations

This avoids building a separate catalogue UI. One product grid, multiple output modes.

---

8. Backend Platform

Module Map (expanded)

Module	Responsibility	Status
`accounts`	**New:** owner signup, account creation, billing plan, multi-brand management	New module
`auth`	Phone OTP, JWT issuance/refresh, session management, owner vs staff login, PIN	From v2 + owner auth
`users`	Staff profiles, emergency contacts	From v2
`roles`	Role definitions, hierarchy	From v2
`capabilities`	Capability profiles, assignment to users/devices	From v2
`devices`	Enrollment, heartbeat, revocation, command inbox	From v2
`stores`	Store config, terminals, printers	From v2
`pos`	Orders, payments, refunds, holds, cart rules, **receipt QR generation (mandatory on every receipt)**	From v2 + receipt QR
`tills`	Open/close, session tracking	From v2
`reconciliation`	Discrepancy workflow, evidence, resolution	From v2
`inventory`	**Expanded:** shelf register, bulk shelf creation, count sessions, device registration, dual-scan protocol, match comparison, dispute handling, label PDF generation	Major expansion
`requests`	Stationery, pickups, customer items — generic request engine	From v2
`workforce`	Attendance, leave, tasks, expenses, asset acceptance, maintenance	From v2
`shifts`	**New:** shift templates, selections, approval, public holiday calendar, attendance QR stations	New module
`loyalty`	**Expanded:** wallets, transactions (order_earn/redeem/survey/staff_issuance/campaign), issuance-pending, points rules, consent (dual-scope, Meta-compliant), vouchers, campaigns	Major expansion
`customers`	**New:** customer profiles, phone normalization (E.164), dedup, registration from POS and WhatsApp	New module
`whatsapp`	**Rewritten:** inbound message handler, WhatsApp Flow triggers, template dispatch, thin SalesIQ relay protocol	Rewritten
`crm-connector`	**New:** one-way push to Zoho CRM on customer/loyalty events for support visibility	New module
`catalogue`	**New:** AI enrichment engine (Claude API), product enrichment queue, review workflow, PDF generation with templates, WhatsApp QR codes, showrooming flow	New module
`reports`	**New:** parameterized SQL views, materialized views, CSV/PDF export, scheduled summaries	New module
`files`	Upload orchestration to Cloudinary, metadata in Postgres	From v2
`notifications`	Push (FCM), in-app, WhatsApp dispatch	From v2
`audit`	Immutable audit event log	From v2
`sync`	Ingest pipeline, transformers, pull delta	From v2
`compliance`	Data deletion, consent audit, DPA 2017 reporting	From v2
`agent-control`	Agent tasks, commands, approval gates	From v2
`jobs`	BullMQ job definitions, scheduling, retry policies	From v2
`chat`	**New:** chat threads (direct/group/AI), message storage, AI assistant with tool-use against existing API endpoints	New module
`logistics`	**New:** shipments, packages, delivery templates, driver assignment, package label generation, receipt confirmation	New module
`operational-supplies`	**New:** non-resale inventory management, reorder alerts, warehouse dispatch — uses `product` table with `product_class='operational'`	New module
`qr-router`	**New:** QR payload generation + deep link registry — single endpoint to generate any QR type	New module
`warehouse`	**New:** container receiving, document vault, inspection workflow, cost allocation, purchase orders, claims, flexible release-to-store	New module

⚠ CONCERN: Module count is high

28 modules is a lot for a modular monolith. Risk of over-engineering the boundaries early.

Recommendation: Start with logical groupings. `customers` + `loyalty` + `whatsapp` could be one `customer-engagement` module initially. `shifts` could live inside `workforce`. Split when the code gets unwieldy, not before.

NestJS directory structure unchanged from v2 §8 — just add the new module directories.

9. Source-of-Truth Data Model

Core Entities (v2 + expanded)

Organization & Structure: `Organization`, `Brand`, `Store`, `Terminal`, `Printer`, `Shelf`

Identity & Access: `User`, `Role`, `CapabilityProfile`, `Device`, `DeviceAssignment`, `DeviceHeartbeat`, `RevocationVersion`, `CapabilitySnapshot`

POS & Transactions: `Order`, `OrderLine`, `Payment`, `TillSession`, `TillReconciliation`, `ReconciliationEvidence`

Customer & Loyalty: `Customer`, `LoyaltyWallet`, `LoyaltyTransaction`, `LoyaltyIssuancePending`, `ConsentRecord`, `Voucher`, `Campaign`

Inventory: `Product`, `Category`, `Barcode`, `InventoryCountSession`, `InventoryCountDevice`, `InventoryShelfCount`, `InventoryShelfCountLine`, `InventoryShelfMatch`, `StockEvent`

Requests & Workflow: `Request`, `Task`, `Approval`, `ApprovalRequest`

Workforce: `LeaveRequest`, `AttendanceEvent`, `ExpenseClaim`, `Asset`, `MaintenanceTicket`

Shifts: `ShiftTemplate`, `PublicHoliday`, `ShiftSelection`, `AttendanceLog`

Compliance: `DataDeletionRequest`

Files & Audit: `Attachment`, `AuditEvent`

Agent: `AgentTask`, `AgentCommand`

Key Schema Design Decisions

1. **UUIDs everywhere.** All primary keys are UUIDv7 (time-sortable). No auto-increment integers exposed externally.
2. **Soft deletes on business entities.** `deleted_at` timestamp, never hard delete in application code.
3. **Immutable orders.** Once `completed`, only refundable via a linked refund order.
4. **Temporal columns on all tables:** `created_at`, `updated_at`, `deleted_at`, `created_by`, `updated_by`.
5. **`idempotency_key` on all mutation-accepting tables** to support offline push replay.
6. **RLS policies** enforce tenant isolation at the Supabase level.
7. **Sync direction is configuration, not code.**
8. **`store_id` on all store-scoped entities.**
9. **DPA compliance built in.** Consent records are append-only with full audit trail.
10. **Wallet balance is denormalized.** `loyalty_wallet.balance` = `SUM(points_change)` from transactions. Nightly reconciliation job verifies.
11. **Consent is dual-scope.** Brand-level AND product-level, with separate promo/news flags per Meta requirements.
12. **Inventory count is dual-verified.** Every shelf requires 2 independent scans that must match.

Customer & Loyalty Schema

```
-- ═══════════ CUSTOMER ═══════════
CREATE TABLE customer (
    id              UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    phone           TEXT NOT NULL UNIQUE,       -- E.164: +23054239978
    phone_plain     TEXT GENERATED ALWAYS AS (regexp_replace(phone, '^\+', '')) STORED,
    name            TEXT NOT NULL DEFAULT 'Customer',
    email           TEXT,
    whatsapp_name   TEXT,
    source          TEXT NOT NULL DEFAULT 'pos'
                    CHECK (source IN ('pos', 'whatsapp', 'web', 'qr_scan', 'import')),
    store_id        UUID REFERENCES store(id),
    created_at      TIMESTAMPTZ NOT NULL DEFAULT now(),
    updated_at      TIMESTAMPTZ NOT NULL DEFAULT now(),
    deleted_at      TIMESTAMPTZ,
    created_by      UUID REFERENCES "user"(id)
);

-- ═══════════ LOYALTY WALLET ═══════════
CREATE TABLE loyalty_wallet (
    id                  UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    customer_id         UUID NOT NULL REFERENCES customer(id),
    organization_id     UUID NOT NULL REFERENCES organization(id),
    balance             INTEGER NOT NULL DEFAULT 0,
    lifetime_earned     INTEGER NOT NULL DEFAULT 0,
    lifetime_redeemed   INTEGER NOT NULL DEFAULT 0,
    status              TEXT NOT NULL DEFAULT 'active'
                        CHECK (status IN ('active', 'suspended', 'closed')),
    created_at          TIMESTAMPTZ NOT NULL DEFAULT now(),
    updated_at          TIMESTAMPTZ NOT NULL DEFAULT now(),
    UNIQUE(customer_id, organization_id)
);

-- ═══════════ LOYALTY TRANSACTION LEDGER ═══════════
CREATE TABLE loyalty_transaction (
    id                  UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    wallet_id           UUID NOT NULL REFERENCES loyalty_wallet(id),
    customer_id         UUID NOT NULL REFERENCES customer(id),
    transaction_type    TEXT NOT NULL
                        CHECK (transaction_type IN (
                            'order_earn', 'redeem', 'survey', 'staff_issuance',
                            'campaign_award', 'voucher_redeem', 'adjustment', 'expiry'
```

```
                        )),
    points_change       INTEGER NOT NULL,       -- positive = earn, negative = spend
    reference_code      TEXT NOT NULL,           -- SO-2026-000985, RED-2026-000030, etc.
    order_id            UUID REFERENCES "order"(id),
    store_id            UUID REFERENCES store(id),
    channel             TEXT NOT NULL DEFAULT 'pos'
                        CHECK (channel IN ('pos', 'whatsapp', 'web', 'system')),
    description         TEXT,
    idempotency_key     UUID UNIQUE,
    created_at          TIMESTAMPTZ NOT NULL DEFAULT now(),
    created_by          UUID REFERENCES "user"(id)
);

-- ========== ISSUANCE PENDING (staff → customer claim) ==========
CREATE TABLE loyalty_issuance_pending (
    id              UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    wallet_id       UUID NOT NULL REFERENCES loyalty_wallet(id),
    customer_id     UUID NOT NULL REFERENCES customer(id),
    points          INTEGER NOT NULL CHECK (points > 0),
    issued_by       UUID NOT NULL REFERENCES "user"(id),
    store_id        UUID NOT NULL REFERENCES store(id),
    claim_token     TEXT NOT NULL UNIQUE,
    status          TEXT NOT NULL DEFAULT 'pending'
                    CHECK (status IN ('pending', 'claimed', 'expired', 'cancelled')),
    expires_at      TIMESTAMPTZ NOT NULL,
    claimed_at      TIMESTAMPTZ,
    transaction_id  UUID REFERENCES loyalty_transaction(id),
    created_at      TIMESTAMPTZ NOT NULL DEFAULT now()
);

-- ========== CONSENT (Meta-compliant, dual-scope, append-only) ==========
CREATE TABLE consent_record (
    id              UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    customer_id     UUID NOT NULL REFERENCES customer(id),
    scope           TEXT NOT NULL DEFAULT 'brand'
                    CHECK (scope IN ('brand', 'product')),
    scope_ref       TEXT,                       -- NULL for brand, product_code for product
    promo_consent   BOOLEAN NOT NULL DEFAULT false,
    news_consent    BOOLEAN NOT NULL DEFAULT false,
    consent_status  TEXT NOT NULL DEFAULT 'none'
                    CHECK (consent_status IN ('none', 'subscribed', 'unsubscribed')),
    source          TEXT NOT NULL,
    granted_at      TIMESTAMPTZ,
    withdrawn_at    TIMESTAMPTZ,
    created_at      TIMESTAMPTZ NOT NULL DEFAULT now(),
    updated_at      TIMESTAMPTZ NOT NULL DEFAULT now()
);

-- ========== VOUCHER ==========
CREATE TABLE voucher (
    id              UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    organization_id UUID NOT NULL REFERENCES organization(id),
    customer_id     UUID REFERENCES customer(id),
    campaign_id     UUID REFERENCES campaign(id),
    code            TEXT NOT NULL UNIQUE,
    voucher_type    TEXT NOT NULL
                    CHECK (voucher_type IN ('fixed_discount', 'percent_discount', 'free_item', 'points_multiplier')),
    value           NUMERIC(12,2) NOT NULL,
    min_spend       NUMERIC(12,2) DEFAULT 0,
    valid_from      TIMESTAMPTZ NOT NULL DEFAULT now(),
    valid_until     TIMESTAMPTZ NOT NULL,
    status          TEXT NOT NULL DEFAULT 'active'
                    CHECK (status IN ('active', 'redeemed', 'expired', 'cancelled')),
    redeemed_at     TIMESTAMPTZ,
    redeemed_order_id UUID REFERENCES "order"(id),
    created_at      TIMESTAMPTZ NOT NULL DEFAULT now()
);

-- ========== CAMPAIGN ==========
CREATE TABLE campaign (
    id              UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    organization_id UUID NOT NULL REFERENCES organization(id),
    code            TEXT NOT NULL UNIQUE,
    name            TEXT NOT NULL,
    campaign_type   TEXT NOT NULL
                    CHECK (campaign_type IN ('voucher', 'survey', 'points_bonus', 'announcement')),
    reward_type     TEXT CHECK (reward_type IN ('fixed_discount', 'percent_discount', 'points', 'free_item', NULL)),
    reward_value    NUMERIC(12,2),
    start_date      TIMESTAMPTZ,
    end_date        TIMESTAMPTZ,
    status          TEXT NOT NULL DEFAULT 'draft'
                    CHECK (status IN ('draft', 'active', 'paused', 'completed', 'cancelled')),
    audience_scope  TEXT NOT NULL DEFAULT 'brand'
                    CHECK (audience_scope IN ('brand', 'promo_consented', 'product_consented', 'all_customers')),
    estimated_reach INTEGER DEFAULT 0,
    created_at      TIMESTAMPTZ NOT NULL DEFAULT now(),
    updated_at      TIMESTAMPTZ NOT NULL DEFAULT now()
);
```

Loyalty Points Rules (C8: configurable per brand)

Points earning rate is configured per organization/brand, stored in an `organization_loyalty_config` table:

```
CREATE TABLE organization_loyalty_config (
    id                   UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    organization_id      UUID NOT NULL REFERENCES organization(id) UNIQUE,
    points_per_currency_unit NUMERIC(8,4) NOT NULL DEFAULT 0.01,  -- e.g. 0.01 = 1pt per Rs100
    currency             TEXT NOT NULL DEFAULT 'MUR',
    min_points_per_txn   INTEGER NOT NULL DEFAULT 1,              -- never award 0
    signup_bonus         INTEGER NOT NULL DEFAULT 100,
    survey_reward        INTEGER NOT NULL DEFAULT 20,
    enable_expiry        BOOLEAN NOT NULL DEFAULT false,
    expiry_months        INTEGER DEFAULT 12,
    created_at           TIMESTAMPTZ NOT NULL DEFAULT now(),
    updated_at           TIMESTAMPTZ NOT NULL DEFAULT now()
);
```

Event	Points	Reference Format	Channel
Purchase at POS	`floor(order_total * points_per_currency_unit)`, min `min_points_per_txn`	`SO-{year}-{seq6}`	pos
Signup bonus	Per org config (default 100)	`SIGNUP-{year}-{seq4}`	pos/whatsapp
WhatsApp survey	Per org config (default 20)	`SURVEY-{year}-{seq4}`	whatsapp
Staff manual issuance	Variable	`ISS-{code}`	pos
Campaign award	Per campaign config	`CAMP-{code}-{seq}`	system
Redemption	Negative	`RED-{year}-{seq6}`	pos/whatsapp

Inventory Count Schema

```sql
-- ═══════════ SHELF REGISTER ═══════════
CREATE TABLE shelf (
    id              UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    store_id        UUID NOT NULL REFERENCES store(id),
    zone_code       TEXT NOT NULL,              -- '001' to '999'
    shelf_number    TEXT NOT NULL,              -- '001' to '999'
    position        TEXT NOT NULL DEFAULT '',   -- '', 'A', 'B', 'C', etc.
    barcode_data    TEXT NOT NULL UNIQUE,       -- 'SHELF|GB|003|012|B'
    label_printed   BOOLEAN NOT NULL DEFAULT false,
    display_name    TEXT,
    active          BOOLEAN NOT NULL DEFAULT true,
    created_at      TIMESTAMPTZ NOT NULL DEFAULT now(),
    UNIQUE(store_id, zone_code, shelf_number, position)
);

-- ═══════════ COUNT SESSION ═══════════
CREATE TABLE inventory_count_session (
    id              UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    store_id        UUID NOT NULL REFERENCES store(id),
    name            TEXT NOT NULL,
    status          TEXT NOT NULL DEFAULT 'planning'
                    CHECK (status IN ('planning', 'active', 'review', 'completed', 'cancelled')),
    initiated_by    UUID NOT NULL REFERENCES "user"(id),
    started_at      TIMESTAMPTZ,
    completed_at    TIMESTAMPTZ,
    created_at      TIMESTAMPTZ NOT NULL DEFAULT now(),
    updated_at      TIMESTAMPTZ NOT NULL DEFAULT now()
);

-- ═══════════ DEVICE REGISTRATION PER SESSION ═══════════
CREATE TABLE inventory_count_device (
    id              UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    session_id      UUID NOT NULL REFERENCES inventory_count_session(id),
    device_id       UUID NOT NULL REFERENCES device(id),
    user_id         UUID NOT NULL REFERENCES "user"(id),
    status          TEXT NOT NULL DEFAULT 'registered'
                    CHECK (status IN ('registered', 'counting', 'done')),
    zones_assigned  TEXT[],
    shelves_completed INTEGER NOT NULL DEFAULT 0,
    shelves_total    INTEGER NOT NULL DEFAULT 0,
    registered_at   TIMESTAMPTZ NOT NULL DEFAULT now()
);

-- ═══════════ SHELF COUNT (one per shelf per scan) ═══════════
CREATE TABLE inventory_shelf_count (
    id              UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    session_id      UUID NOT NULL REFERENCES inventory_count_session(id),
    shelf_id        UUID NOT NULL REFERENCES shelf(id),
    device_id       UUID NOT NULL REFERENCES device(id),
    user_id         UUID NOT NULL REFERENCES "user"(id),
    scan_number     INTEGER NOT NULL DEFAULT 1,   -- 1=first, 2=second, 3=tiebreak
    status          TEXT NOT NULL DEFAULT 'open'
                    CHECK (status IN ('open', 'closed', 'disputed')),
    opened_at       TIMESTAMPTZ NOT NULL DEFAULT now(),
    closed_at       TIMESTAMPTZ,
    idempotency_key UUID UNIQUE,
    UNIQUE(session_id, shelf_id, scan_number)
);

-- ═══════════ COUNT LINE (products on shelf) ═══════════
CREATE TABLE inventory_shelf_count_line (
    id              UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    shelf_count_id  UUID NOT NULL REFERENCES inventory_shelf_count(id),
    product_id      UUID NOT NULL REFERENCES product(id),
    barcode_scanned TEXT NOT NULL,
    quantity        INTEGER NOT NULL DEFAULT 1,
    scan_method     TEXT NOT NULL DEFAULT 'barcode'
                    CHECK (scan_method IN ('barcode', 'qr', 'manual_qty')),
    scanned_at      TIMESTAMPTZ NOT NULL DEFAULT now()
);

-- ═══════════ SHELF MATCH RESULT ═══════════
CREATE TABLE inventory_shelf_match (
    id              UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    session_id      UUID NOT NULL REFERENCES inventory_count_session(id),
    shelf_id        UUID NOT NULL REFERENCES shelf(id),
    scan_1_id       UUID NOT NULL REFERENCES inventory_shelf_count(id),
    scan_2_id       UUID NOT NULL REFERENCES inventory_shelf_count(id),
    scan_3_id       UUID REFERENCES inventory_shelf_count(id),
    match_status    TEXT NOT NULL DEFAULT 'pending'
                    CHECK (match_status IN ('pending', 'matched', 'disputed', 'resolved')),
    mismatched_products JSONB,
    resolved_by     UUID REFERENCES "user"(id),
    resolved_at     TIMESTAMPTZ,
    created_at      TIMESTAMPTZ NOT NULL DEFAULT now()
);
```

Inventory Count Protocol

Shelf addressing: `{STORE_CODE}-{ZONE}-{SHELF}{POSITION}` e.g. `GB-003-012B`

Barcode encoding: Code 128: `SHELF|GB|003|012|B` or QR with JSON: `{"t":"shelf","s":"GB","z":"003","n":"012","p":"B"}`

Counting flow:

1. Scan shelf barcode → server opens `inventory_shelf_count` (scan_number=1)
2. Scan product barcodes → create count lines (qty increments on duplicate scan)
3. Scan shelf barcode again → server closes the shelf count
4. Different device scans same shelf → server opens scan_number=2

5. After both scans close → server compares per-product quantities
6. All match → `matched`. Any mismatch → `disputed` → supervisor assigns 3rd device → majority wins

Enforcement rules (full count): - At least 1 barcode scan per shelf (reject close if zero lines) - At least 2 closed scans per shelf before session can complete - Shelf must be closed before opening next (device-enforced) - Cannot scan a shelf already open by another device

Spot Check Mode (C5 resolved)

Spot checks are a lighter-weight continuous inventory control mechanism. They don't replace full counts — they generate KPIs that tell you WHEN a full count is needed.

How spot checks work: - Any authorized staff can initiate a spot check (no session creation, no device registration) - Single scan only — no dual verification required - Staff scans a shelf → scans products → closes shelf - Result is recorded but NOT compared against expected stock (because items may be elsewhere in the store) - Spot check data feeds a **Shelf Accuracy KPI:** - Compare spot check qty vs last full count qty for that shelf - If variance exceeds threshold (e.g. >15% discrepancy on 3+ shelves in a zone), flag zone for full count - Dashboard shows: zones that are "drifting" from last full count

What spot checks CANNOT tell you: - Total store inventory accuracy (items may be on wrong shelves, in back room, etc.) - Shrinkage (need a full count for that)

What spot checks CAN tell you: - Which zones are getting messy (merchandise displacement) - Which products are selling faster than expected (shelf depletion rate) - When the next full count should be scheduled - Staff accountability (who checked what, when)

Spot check schema addition:

```
ALTER TABLE inventory_count_session ADD COLUMN session_type TEXT NOT NULL DEFAULT 'full_count'
    CHECK (session_type IN ('full_count', 'spot_check'));
-- Spot checks: scan_number always 1, no match comparison, no dual-scan enforcement
-- Full counts: existing dual-scan protocol applies
```

Spot check report: "Shelf Accuracy Drift" — shows variance between spot check and last full count per zone, ranked by drift magnitude. When a zone's drift score exceeds the configured threshold, the report recommends scheduling a full count.

Shelf Label Barcode Format Options (C6 resolved)

Multiple barcode formats supported. Format is configured per store (or globally):

Format	Encoding	Best For	Label Size
Code 128	`SHELF|GB|003|012|B`	Epson receipt printers, fast linear scan	80mm × 30mm
QR Code	`{"t":"shelf","s":"GB","z":"003","n":"012","p":"B"}`	Camera scanning, more data capacity	40mm × 40mm
Code 39	`GB003012B`	Zebra label printers, wide compatibility	100mm × 30mm
DataMatrix	Compact binary encoding	Small labels, high density	20mm × 20mm

Recommendation: Start with QR Code (maximum flexibility — works with phone cameras and external scanners, carries full structured data). Add Code 128 as a secondary option for stores that prefer linear barcodes with external scanners.

Shelf label printing (C7: Zebra + Epson + personal):

Printer	Method	Label Quality
Zebra label printer	ZPL commands via Bluetooth/USB	Best — purpose-built for labels, adhesive stock, weather-resistant
Epson receipt printer	ESC/POS commands, receipt paper	Good — quick and available, but not adhesive
Personal/office printer	PDF generation → Android Print Framework	Basic — paper labels, need to cut and tape

Endpoint `POST /v1/inventory/shelves/labels` accepts: array of shelf IDs + format (qr/code128/code39) + printer_type (zebra/epson/pdf). Returns ZPL commands, ESC/POS commands, or PDF respectively.

Bulk shelf creation: `POST /v1/inventory/shelves/bulk` — accepts store, zone range, shelf range, positions → creates all shelf records.

Shift Planning Schema

```sql
CREATE TABLE shift_template (
    id              UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    store_id        UUID NOT NULL REFERENCES store(id),
    shift_code      TEXT NOT NULL UNIQUE,
    shift_name      TEXT NOT NULL,
    shift_number    INTEGER NOT NULL,
    role_code       TEXT NOT NULL,
    day_type        TEXT NOT NULL CHECK (day_type IN ('weekday', 'weekend', 'public_holiday')),
    applies_on      TEXT NOT NULL DEFAULT 'all',
    start_time      TIME NOT NULL,
    end_time        TIME NOT NULL,
    max_staff_count INTEGER NOT NULL DEFAULT 1,
    requires_approval BOOLEAN NOT NULL DEFAULT true,
    active          BOOLEAN NOT NULL DEFAULT true,
    created_at      TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE TABLE public_holiday (
    id              UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    country         TEXT NOT NULL DEFAULT 'Mauritius',
    holiday_code    TEXT NOT NULL UNIQUE,
    holiday_name    TEXT NOT NULL,
    holiday_date    DATE NOT NULL,
    date_status     TEXT NOT NULL DEFAULT 'confirmed'
                    CHECK (date_status IN ('confirmed', 'subject_to_confirmation')),
    notes           TEXT,
    active          BOOLEAN NOT NULL DEFAULT true
);

CREATE TABLE shift_selection (
    id                  UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    shift_template_id   UUID NOT NULL REFERENCES shift_template(id),
    user_id             UUID NOT NULL REFERENCES "user"(id),
    shift_date          DATE NOT NULL,
    plan_week_start     DATE NOT NULL,
    selection_code      TEXT NOT NULL UNIQUE,
    status              TEXT NOT NULL DEFAULT 'pending'
                        CHECK (status IN ('pending', 'approved', 'rejected', 'cancelled')),
    submitted_at        TIMESTAMPTZ NOT NULL DEFAULT now(),
    approved_at         TIMESTAMPTZ,
    approved_by         UUID REFERENCES "user"(id),
    manager_note        TEXT,
    holiday_name        TEXT
);

CREATE TABLE attendance_log (
    id                  UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    shift_selection_id  UUID REFERENCES shift_selection(id),
    user_id             UUID NOT NULL REFERENCES "user"(id),
    store_id            UUID NOT NULL REFERENCES store(id),
    scan_direction      TEXT NOT NULL CHECK (scan_direction IN ('time_in', 'time_out')),
    scan_timestamp      TIMESTAMPTZ NOT NULL,
    scan_source         TEXT NOT NULL DEFAULT 'qr'
                        CHECK (scan_source IN ('qr', 'manual', 'gps', 'nfc')),
    qr_token            TEXT,
    attendance_code     TEXT NOT NULL UNIQUE,
    status              TEXT NOT NULL DEFAULT 'captured',
    idempotency_key     UUID UNIQUE,
    created_at          TIMESTAMPTZ NOT NULL DEFAULT now()
);
```

v2 schema design decisions, data storage rules, and all other §9 content remain in effect.

---

10. Data Migration Strategy (simplified)

Since Zoho Creator and CRM contain only playground/test data, migration is dramatically simpler than v2 described.

What Needs to Happen

Step	Action	Effort
Supabase schema deployment	Run migration scripts against fresh database	Automated
Reference data seeding	Products, categories, stores, terminals from CSV or manual entry	1-2 days
Staff records	Users, roles, capabilities from CSV or manual entry	1 day
Public holidays	Seed 15 Mauritius 2026 holidays (data already available from portal)	Minutes
Shelf register	Bulk creation via API once store layout is mapped	Per-store setup
Android app transition	Incremental: wrap legacy → add shell → swap endpoints	Phased over weeks

What Does NOT Need to Happen

- ~~Migrate loyalty wallets from Creator~~ (only 2 test wallets)
- ~~Migrate customer contacts from CRM~~ (test data)
- ~~Migrate transaction history from Flask~~ (proxy layer, no local data)
- ~~Dual-write period~~ (no legacy system to keep in sync)
- ~~Data reconciliation reports~~ (nothing to reconcile)

⚠ CONCERN: Android POS historical data

I haven't seen the Android POS codebase. There may be historical transaction data in the Android Room database or in whatever backend the current app talks to.

Question for Fred: Does the current POS Android app store order history anywhere besides the local Room database? Is there a server it syncs to today?

---

11. API Conventions

Unchanged from v2 §11. Wire format (snake_case, /v1/ prefix), response envelopes, error codes, cursor pagination, idempotency, rate limiting all remain as specified.

---

12. API Route Catalog (expanded)

Auth, Devices, Capabilities, POS, Tills, Products, Categories, Barcodes, Reconciliation, Requests, Workforce, Files, Agent Control, Sync, Compliance

All routes from v2 §12 remain unchanged.

Accounts & Onboarding (new)

- `POST /v1/accounts/signup` — Owner signup (phone → create owner + account)
- `POST /v1/accounts/verify-otp` — Verify owner phone OTP
- `POST /v1/accounts/complete-profile` — Set owner name + create PIN
- `POST /v1/accounts/brands` — Create brand (name, category, location)
- `GET /v1/accounts/brands` — List owner's brands
- `POST /v1/accounts/brands/{id}/stores` — Create store under brand
- `GET /v1/accounts/brands/{id}/stores` — List stores for brand
- `POST /v1/accounts/brands/{id}/ai-products` — Trigger AI starter product generation
- `GET /v1/accounts/brands/{id}/ai-products` — List AI-generated products pending review
- `POST /v1/accounts/ai-products/{id}/accept` — Accept AI product → moves to master
- `POST /v1/accounts/ai-products/{id}/edit` — Accept with edits → moves to master
- `POST /v1/accounts/ai-products/{id}/skip` — Skip/discard AI product
- `POST /v1/accounts/ai-products/skip-all` — Skip all remaining
- `POST /v1/auth/owner-login` — Owner login (PIN/biometric → 30-day refresh JWT)
- `POST /v1/auth/staff-login` — Staff login (staff picker + PIN → 7-day refresh JWT)
- `GET /v1/auth/staff-list` — List staff assigned to this device's store (for login picker)

Inventory (expanded)

- `POST /v1/inventory/shelves/bulk` — Bulk create shelf records (zone range, shelf range, positions)
- `POST /v1/inventory/shelves/labels` — Generate label PDF for shelf IDs
- `GET /v1/inventory/shelves` — List shelves for store (filterable by zone)
- `POST /v1/inventory/count-sessions` — Create count session
- `POST /v1/inventory/count-sessions/{id}/start` — Start session (move to active)
- `POST /v1/inventory/count-sessions/{id}/devices` — Register device to session
- `POST /v1/inventory/count-sessions/{id}/complete` — Complete session (requires all shelves dual-verified)
- `POST /v1/inventory/shelf-counts/open` — Scan shelf to open count
- `POST /v1/inventory/shelf-counts/{id}/lines` — Add product line to open shelf count
- `POST /v1/inventory/shelf-counts/{id}/close` — Scan shelf to close count
- `GET /v1/inventory/count-sessions/{id}/progress` — Session progress (per zone, per device)
- `GET /v1/inventory/count-sessions/{id}/matches` — Match results and disputes
- `POST /v1/inventory/shelf-matches/{id}/assign-tiebreak` — Assign 3rd device for disputed shelf
- `POST /v1/inventory/shelf-matches/{id}/resolve` — Manager manual resolution

Customers (new)

- `POST /v1/customers` — Create customer (phone normalization, dedup check)
- `GET /v1/customers/lookup?phone={phone}` — Lookup by phone (used by POS and WhatsApp)
- `GET /v1/customers/{id}` — Customer detail with wallet and consent
- `PUT /v1/customers/{id}` — Update customer profile
- `DELETE /v1/customers/{id}` — Soft delete (DPA right to erasure)

Loyalty (expanded)

- `GET /v1/loyalty/wallets/{phone}` — Look up wallet by phone
- `GET /v1/loyalty/wallets/{id}/transactions` — Transaction history
- `POST /v1/loyalty/award` — Award points (idempotent by order UUID)
- `POST /v1/loyalty/redeem` — Redeem points
- `POST /v1/loyalty/issuance` — Staff creates pending issuance
- `POST /v1/loyalty/issuance/{id}/claim` — Customer claims pending issuance
- `POST /v1/loyalty/consent` — Record/update consent (append-only audit)
- `GET /v1/loyalty/consent/{customer_id}` — Current consent state
- `POST /v1/loyalty/vouchers` — Issue voucher
- `POST /v1/loyalty/vouchers/validate` — Check voucher validity
- `POST /v1/loyalty/vouchers/redeem` — Redeem voucher (atomic validate + deduct)
- `POST /v1/loyalty/campaigns` — Create campaign
- `PUT /v1/loyalty/campaigns/{id}` — Update campaign
- `POST /v1/loyalty/campaigns/{id}/activate` — Activate campaign
- `GET /v1/loyalty/campaigns/{id}/reach` — Estimate audience reach

Shifts (new)

- `POST /v1/shifts/templates` — Create shift template
- `GET /v1/shifts/templates` — List templates for store
- `POST /v1/shifts/selections` — Staff selects a shift
- `POST /v1/shifts/selections/{id}/approve` — Manager approves
- `POST /v1/shifts/selections/{id}/reject` — Manager rejects
- `GET /v1/shifts/schedule?week_start={date}` — Weekly schedule view
- `GET /v1/shifts/holidays` — Public holiday calendar

WhatsApp (new)

- `POST /v1/whatsapp/inbound` — Receive message from SalesIQ relay
- `POST /v1/whatsapp/flows/registration` — Trigger registration WhatsApp Flow
- `POST /v1/whatsapp/flows/survey` — Trigger survey WhatsApp Flow
- `POST /v1/whatsapp/templates/send` — Send approved template message
- `GET /v1/whatsapp/templates` — List approved Meta templates

Reports (new)

- `GET /v1/reports/daily-sales?store_id=&date=` — Daily sales summary
- `GET /v1/reports/hourly-heatmap?store_id=&date=` — Hourly sales breakdown
- `GET /v1/reports/product-ranking?store_id=&period=` — Product sales ranking
- `GET /v1/reports/discrepancy-summary?store_id=&period=` — Till discrepancy report
- `GET /v1/reports/device-health` — Device fleet health
- `GET /v1/reports/count-session/{id}` — Inventory count results
- `GET /v1/reports/enrollment-funnel?period=` — Loyalty enrollment funnel
- `GET /v1/reports/points-economy?period=` — Points earned vs redeemed
- `GET /v1/reports/voucher-performance?campaign_id=` — Voucher redemption rates
- `GET /v1/reports/attendance-summary?store_id=&period=` — Attendance report
- `POST /v1/reports/export` — Export any report as CSV or PDF

Chat (new)

- `POST /v1/chat/threads` — Create thread (direct/group)
- `GET /v1/chat/threads` — List user's threads with unread counts
- `GET /v1/chat/threads/{id}/messages` — Message history (cursor-paginated)
- `POST /v1/chat/threads/{id}/messages` — Send message
- `POST /v1/chat/threads/{id}/read` — Mark thread as read
- `POST /v1/chat/ai` — Send message to AI assistant (returns AI response + any actions taken)

Logistics (new)

- `POST /v1/logistics/shipments` — Create shipment
- `GET /v1/logistics/shipments` — List shipments (filtered by status, driver, store)
- `GET /v1/logistics/shipments/{id}` — Shipment detail with packages and step progress
- `POST /v1/logistics/shipments/{id}/assign` — Assign driver
- `POST /v1/logistics/shipments/{id}/pickup` — Driver confirms pickup

- `POST /v1/logistics/shipments/{id}/deliver` — Driver confirms delivery complete
- `POST /v1/logistics/packages` — Create package within shipment
- `POST /v1/logistics/packages/{id}/label` — Generate package label (QR)
- `POST /v1/logistics/packages/{id}/scan` — Record package scan event
- `POST /v1/logistics/packages/{id}/receive` — Recipient confirms package received
- `POST /v1/logistics/shipments/{id}/steps/{step_id}/complete` — Complete a delivery template step
- `GET /v1/logistics/templates` — List delivery templates
- `POST /v1/logistics/templates` — Create delivery template
- `POST /v1/logistics/pickup-requests` — Store requests driver pickup
- `POST /v1/logistics/shipments/{id}/cod/collect` — Driver records COD payment (amount, method, photo)
- `GET /v1/logistics/cod/pending` — List undeposited COD cash per driver
- `POST /v1/logistics/cod/{payment_id}/deposit` — Driver records cash deposit (evidence photo)
- `POST /v1/logistics/cod/{payment_id}/reconcile` — Manager confirms reconciliation
- `GET /v1/logistics/cod/report?driver_id=&period=` — COD collection summary report

QR Generation (new)

- `POST /v1/qr/product/{sku}` — Generate product QR (WhatsApp deep link)
- `POST /v1/qr/store/{store_code}` — Generate store welcome QR
- `POST /v1/qr/receipt/{order_ref}` — Generate receipt QR
- `POST /v1/qr/package/{package_id}` — Generate package QR (in-app deep link)
- `POST /v1/qr/attendance/{store_code}/{direction}` — Generate attendance station QR
- `POST /v1/qr/bulk` — Batch generate QR codes

CRM Connector (internal, not exposed as API)

Triggered by internal events via BullMQ: - On customer create/update → upsert Zoho CRM Contact - On loyalty enrollment → set `Posterita_Loyalty_Enrolled = true` - On consent change → set `WhatsApp_Marketing_Consent` + timestamp - On balance change → update `Loyalty_Points` field - On voucher redemption → add CRM Note

---

13. Web Control Console (expanded)

Sections

Section	Primary Users	Key Functions
Dashboard	Supervisors, Managers	KPIs, alerts, pending approvals, live inventory count progress
Devices	IT Admin, Supervisors	Assign, revoke, monitor heartbeat, push commands
Users & Roles	Managers	Invite, assign roles, manage profiles
Capabilities	Managers	Create/edit capability profiles, assign to users/devices
Stores & Terminals	Managers	Store config, terminal setup, printer assignment, shelf register
Products	Managers	Product CRUD, pricing, stock levels, Cloudinary media
Reconciliations	Supervisors	Review discrepancies, view evidence, resolve
Inventory Counts	Supervisors	Create sessions, register devices, live progress dashboard, dispute resolution
Requests	Supervisors	Stationery, pickup, customer item approvals
Staff Ops	Supervisors	Attendance review, leave approval, expense approval
Shifts	Managers	Shift templates, selections, approval, holiday calendar, QR station config
Customers & Loyalty	Managers	Customer lookup, wallet management, voucher management, consent audit
Campaigns	Managers	Create/edit campaigns, audience estimation, delivery preview
Reports	Managers	All report views with filters, export to CSV/PDF
Audit Trail	Managers, Compliance	Searchable audit log with filters
Data Compliance	Managers	Customer data deletion requests, consent audit, DPA reporting
AI Task Center	Managers	Agent task queue, command approvals, execution log
AI Setup	Managers	Configure auto-execute vs approval-required agent actions
Logistics	Managers, Drivers	Shipment management, driver assignment, package tracking, delivery templates
Operational Supplies	Managers	Non-resale inventory, reorder alerts, warehouse dispatch

⚠ CONCERN: 18 sections is a lot for a web console

Not all sections need to ship in MVP. Recommend phased rollout:

MVP (Phase 2-3): Dashboard, Devices, Users & Roles, Stores, Products, Reconciliations, Inventory Counts, Customers & Loyalty, Reports, Audit Trail

Phase 4: Staff Ops, Shifts, Campaigns, AI Task Center, Data Compliance, AI Setup

Post-MVP: Capabilities (merged into Users & Roles initially), Requests (merged into Staff Ops initially)

---

14. UI and UX Direction

Android Home Screen

Unchanged from v2 §14 — role-first home screen, capability-driven tile visibility.

Product Display — Critical Review

The current product display (from the ProductActivity mockup) works but has specific weaknesses:

Problem 1: Cards lack visual differentiation. At 68px with a 56px image strip, products blend together in large catalogues.

Fix: Two view modes via toggle in top bar: - **Compact list** (current default) — for power users who know the catalogue - **Visual grid** — larger cards (120px) with 80px square images, 2 columns

Problem 2: Category chips are undifferentiated text walls.

Fix: Add a 4px left-border color accent on each category chip. Color defined in category record.

Problem 3: Search is buried below categories and order type.

Fix: Add a persistent search icon in the top bar that expands to a search field on tap. Search is the fastest product-finding method for large catalogues — it should be top-level.

Problem 4: No stock visibility on product cards.

Fix: Thin colored bar at bottom of each card: - Green: in stock (≥10) - Amber: low stock (1-9) - Red: out of stock (0) — card greyed, not tappable

Problem 5: No frequently-sold shortcut.

Fix: Auto-populated "★ FREQUENT" category chip showing the 12 most-sold products in the last 7 days. Zero configuration — calculated from transaction data.

⚠ CONCERN: Product count determines UI priority

Question for Fred: How many products does a typical Funky Fish store carry? If it's 30-50, the current compact grid is fine and search is secondary. If it's 200+, search becomes the primary navigation and the grid view toggle becomes essential. The answer determines which improvements are MVP vs later.

UX Principles

Unchanged from v2 §14 — POS fast/touch-first, Staff Ops daily-assistant, Supervisor approval-cockpit, Close Till guided flow.

---

15. Performance Targets

Unchanged from v2 §15. All targets remain: - Cold start <3s, scan-to-cart <200ms, checkout <500ms, sync push <1s, API p95 <300ms, etc.

Addition: | Metric | Target | |---|---| | Shelf scan to open confirmation | < 500ms | | Product scan during count (continuous) | < 200ms between scans | | Inventory count dashboard refresh | < 2 seconds (Realtime subscription) | | Shelf label PDF generation (100 labels) | < 5 seconds |

---

16. Observability and Monitoring

Unchanged from v2 §16. Sentry, Render metrics, UptimeRobot, structured logging, Firebase Crashlytics all remain as specified.

---

17. Hosting, Cost, and Resource Planning

Base infrastructure unchanged from v2 §17.

Updated Monthly Cost Estimate (Production MVP)

Service	Estimate
Render (API + Worker + Redis)	$40–60
Supabase Pro	$25
Cloudinary Plus	$89
Vercel Pro	$20
Anthropic API (AI enrichment)	$5–15 (depends on catalogue size and re-enrichment frequency)
Meta WhatsApp (10K customers/yr, C4)	$20–50 (beyond free 1,000 conversations/month)
Sentry, UptimeRobot, Firebase	$0 (free tiers)
Total	**~$200–260/month**

---

18. Security and Compliance

Unchanged from v2 §18. Access control, audit trail, financial safety, AI safety gates all remain as specified.

Addition to Customer Communications:

- All WhatsApp Flows must collect explicit consent before enrollment
- Consent is dual-scope (brand + product) with separate promo/news flags
- The consent table is append-only — every state change creates a new row for full audit
- Annual consent re-confirmation via WhatsApp template (DPA requirement)
- "STOP" keyword must trigger immediate consent withdrawal and acknowledge within the same conversation turn

---

19. Testing Strategy

v2 §19 testing strategy remains with these additions:

Additional E2E Test Scenarios

Scenario	Coverage
Inventory count: 2 devices scan same shelf, counts match	Inventory dual-scan + match
Inventory count: 2 devices disagree, 3rd device resolves	Dispute + tiebreak
Shelf scan error: scan wrong shelf while another is open	Error handling
Customer registration via WhatsApp Flow	WhatsApp + Customer + Loyalty
Consent grant → check in CRM mirror	WhatsApp + Consent + CRM connector
Award points at POS → check wallet on WhatsApp	POS + Loyalty + WhatsApp
Voucher redeem at POS → verify voucher status	POS + Voucher + atomic transaction
Shelf label print for 50 shelves	Label generation + printer
Campaign activate → audience estimation → send templates	Campaign + WhatsApp templates
AI enrich product → review → accept → verify master record updated	Catalogue + AI + Review workflow
AI enrich with low confidence → reject → re-enrich with more context	Catalogue + AI safety controls
Generate catalogue PDF from approved products	Catalogue + PDF + Cloudinary
Customer scans showroom QR → receives product details on WhatsApp	Showrooming + WhatsApp + Catalogue
Enrolled customer scans receipt QR → sees points confirmation	Receipt QR + Loyalty + WhatsApp
Non-enrolled customer scans receipt QR → registers → retroactive points awarded	Receipt QR + Registration + Retroactive award
Enrolled customer scans receipt for unlinked order → order linked, points awarded	Receipt QR + Retroactive link + Idempotent award
Owner signup → AI generates 25 products → owner accepts 18, skips 7 → 18 in master	Onboarding + AI generation + product review
Owner creates second brand → AI generates products for different category	Multi-brand + AI context switching
Staff login picker → select user → enter PIN → correct store capabilities loaded	Device auth + staff picker + capabilities

Additional Manual UAT

Device Config	Test Focus
2 devices in same count session	Dual-scan workflow, conflict detection
Customer WhatsApp on personal phone	Registration Flow, consent, points check
Shelf label on Epson receipt printer	Label format, barcode scannability

---

20. Risk Register (updated)

Risk	Likelihood	Impact	Mitigation
Offline sync push fails silently	Low	Medium	Server always accepts; device retries indefinitely; outbox size limit with warning
Room wipe loses unpushed outbox	Low	Medium	`onCreate` callback + aggressive push frequency
Blink SDK compatibility	Medium	High	Evaluate from playground code early in Phase 1; if incompatible, plan replacement
Staff resistance to new UI	Medium	Medium	Keep cashier flow familiar; change navigation only first
Supabase Realtime drops	Medium	Low	Web console degrades to polling; Android never depends on Realtime
Render cold starts	Low	Medium	Keep instances warm; paid tier with zero-downtime deploys
Inventory count disputes overwhelm supervisor	Medium	Medium	Good dashboard UX; auto-resolution when 3rd scan agrees with either; batch dispute view
WhatsApp Flow approval delayed by Meta	Medium	High	Build button-based fallback flow alongside Flows; swap seamlessly
WhatsApp conversation costs spike	Low	Medium	Monitor monthly; set budget alerts; throttle non-critical templates
AI enrichment generates inaccurate product specs	High	Low	Human review required on every field; low-confidence flagged; specs prompt says "don't invent"
AI enrichment costs spike on large catalogue	Low	Low	~$0.01/product; 1,000 products = $12.50. Negligible.
Scope creep into deferred features	High	Medium	MVP boundary documented; all deferrals require explicit decision
CRM connector fails silently	Medium	Low	CRM is non-critical; dead letter queue with alerts; support team uses Retail OS directly as fallback
Shelf barcode labels fade/damage	Medium	Medium	Laminated labels; re-print endpoint; shelf can be looked up by zone+number if barcode unreadable
Dual-scan requirement slows inventory counts	Low	Medium	It's deliberate — accuracy over speed. But offer "spot check" mode for daily quick counts (single scan)

⚠ CONCERN: "Spot check" vs "Full count" modes

The dual-scan protocol is warehouse-grade and appropriate for periodic full counts. But you also mentioned "spot checks" as a use case. Spot checks need a lighter-weight mode:

- **Full count:** Dual-scan required, session-based, all shelves, supervisor-initiated
- **Spot check:** Single-scan, no matching required, individual shelf or zone, any authorized staff

Question for Fred: Do you want spot check mode in MVP, or is full count sufficient?

---

21. Implementation Sequence (updated)

Phase 0: Foundations (Weeks 1–3)

- [ ] Create backend repo (NestJS) and web repo (Next.js)
- [ ] Stand up Supabase project: full schema including customer, loyalty, inventory, shifts
- [ ] Confirm Supabase Pro plan with daily backups
- [ ] Stand up Render services: API, worker, cron, Redis
- [ ] Stand up Cloudinary
- [ ] Implement: auth, device, capability, audit, file upload, notification modules
- [ ] Implement: accounts module — owner signup, account creation, organization/brand/store CRUD
- [ ] Implement: AI product generation for onboarding (Claude API → product_enrichment pipeline)
- [ ] Implement: sync module — ingest pipeline, pull delta, transformers
- [ ] Implement: customer module (phone normalization, dedup)
- [ ] Implement: product and category CRUD
- [ ] Create shelf table and bulk creation endpoint
- [ ] Create loyalty schema (wallet, transaction, consent, voucher, campaign)

- [ ] Create shift planning schema + seed Mauritius 2026 holidays
- [ ] Create reports module skeleton with daily sales summary materialized view
- [ ] Deploy monitoring: Sentry, health checks, UptimeRobot
- [ ] Test Supabase backup restore on staging

Phase 1: Android Shell + POS Foundation (Weeks 3–6)

- [ ] **Evaluate playground codebase:** review existing Android POS code, identify reusable components (layouts, Blink wiring, scanner handling, navigation patterns)
- [ ] **Decision gate:** for each existing component, decide: adopt as-is, adapt, or rebuild
- [ ] Apply for Meta WhatsApp Flows access (long lead time — start now)
- [ ] Build: `:app` shell, `:core:designsystem` (brand tokens), `:core:auth`, `:core:device`
- [ ] Build: `:core:printer` (Epson ePOS + Zebra ZPL abstraction, receipts + labels)
- [ ] Build: `:core:scanner` (camera + Bluetooth HID, configurable modes)
- [ ] Build: `:core:sync` (outbox push + server pull + Room strategy)
- [ ] Build: `:core:network` (API client, connectivity observer)
- [ ] Build: `:feature:onboarding` — multi-step owner signup (phone → OTP → name → brand → location → category → AI products → review → PIN)
- [ ] Build: `:feature:home` (role-first, capability-driven tile visibility, brand switcher for owners)
- [ ] Build: `:feature:pos` — product grid, cart, checkout, payment (adopt from playground where good)
- [ ] Build: receipt printing with **mandatory QR code** on every receipt (ESC/POS `addSymbol()`, min 25mm, QRCODE_MODEL_2)
- [ ] Implement: receipt QR format — `wa.me/{number}?text=RECEIPT%20{ORDER_REF}` encoded as QR on receipt footer
- [ ] Implement: auth flow — owner login (PIN/biometric, 30-day refresh) AND staff login (staff picker → PIN, 7-day refresh)
- [ ] Implement: device enrollment (QR scan)
- [ ] Implement: Blink payment integration (adopt from playground if compatible)

Phase 2: Loyalty + Inventory + WhatsApp + QR + AI Chat (Weeks 6–11)

- [ ] Implement: loyalty engine (award, balance, redeem — preserving Flask API contract from §34)
- [ ] Implement: customer registration from POS (thin client) and WhatsApp
- [ ] Implement: WhatsApp module — thin Zobot relay + backend inbound handler
- [ ] Implement: Universal QR deep router (`:core:navigation` — parse payload, route to correct screen)
- [ ] Implement: QR code generation endpoints (product, store, receipt, shelf, package, attendance, bulk)
- [ ] Implement: WhatsApp inbound trigger routing (DETAILS, WELCOME, JOIN, RECEIPT, HI)
- [ ] Implement: RECEIPT trigger — retroactive point award for unlinked orders
- [ ] Implement: customer lookup-or-create pipeline (phone normalization → lookup → create → registration Flow)
- [ ] Implement: QR scan event logging + showroom funnel analytics
- [ ] Implement: WhatsApp Flows (registration + consent) — **if Meta approved; button fallback otherwise**
- [ ] Implement: CRM connector (one-way push for support visibility)
- [ ] Implement: inventory count full protocol (session, device registration, dual-scan, match, dispute)
- [ ] Implement: spot check mode (single-scan, shelf accuracy KPI)
- [ ] Implement: shelf label printing (Zebra ZPL + Epson ESC/POS + PDF)
- [ ] Implement: `:feature:catalogue` (reuse POS grid → PDF catalogue, barcode labels)
- [ ] Implement: AI enrichment engine (Claude API integration, product_enrichment table, batch queue)
- [ ] Implement: enrichment review UI in web console (per-field accept/reject/edit, confidence display)
- [ ] Implement: **AI assistant chat** (`:feature:chat` MVP — Claude API with tool-use against existing backend endpoints, scoped to user permissions)
- [ ] Implement: **operational supplies** — add `product_class` field, operational supply categories, reorder alerts
- [ ] Implement: **basic logistics** — shipments, packages, labels, standard parcel template, package scanning
- [ ] Implement: reconciliation workflow
- [ ] Implement: request engine
- [ ] Wire sync engine end-to-end
- [ ] Web console: dashboard, devices, users, stores, products (with operational supplies filter), reconciliation, inventory count live dashboard, customer/loyalty management, AI chat

Phase 3: Staff + Shifts + Reports + Campaigns + Full Logistics + Chat (Weeks 11–15)

- [ ] Implement: workforce module (attendance, leave, tasks, expenses)
- [ ] Implement: shift planning (templates, selections, approval, attendance QR)
- [ ] Implement: attendance QR stations — QR at store entrance, auto-submit on scan
- [ ] Implement: campaign engine (create, activate, audience, dispatch)
- [ ] Implement: voucher management (issue, redeem, expiry cron)
- [ ] Implement: reports (MVP set: daily sales, discrepancy, device health, count results, enrollment funnel, showroom funnel)
- [ ] Implement: catalogue PDF generation (templates: credit card, A5, compact, flyer)

- [ ] Implement: **direct + group chat** (`:feature:chat` expansion — threads, members, unread badges)
- [ ] Implement: **full delivery templates** (motorcycle handover, inter-store transfer, pickup request)
- [ ] Implement: **driver assignment workflow** — assign driver to shipment, driver sees assigned jobs
- [ ] Implement: **store-to-store transfer** — request → approve → driver picks up → recipient scans to receive → inventory adjusts
- [ ] Implement: **warehouse dispatch** for operational supplies — request → pick → ship → receive
- [ ] Implement: **container receiving** — container record, document vault, inspection workflow, photo capture
- [ ] Implement: **flexible release-to-store** — inspect box → create/link products (selling price now) → release via logistics → cost later
- [ ] Implement: **cost allocation engine** — landed cost calculation, proportional allocation, PO generation, reconciliation check
- [ ] Implement: **claims workflow** — create claim, link evidence, track resolution
- [ ] Implement: **"Barcode My Store"** — guided shelf-by-shelf capture + AI product ID + label printing in shelf order
- [ ] Build: Android `:feature:staff-ops`, `:feature:supervisor`, `:feature:logistics`, `:feature:warehouse`, `:feature:barcode-my-store`
- [ ] Web console: staff ops, shifts, campaigns, reports, audit trail, compliance, AI task center, logistics dashboard, containers, purchase orders, claims
- [ ] Real-time inventory count dashboard (Supabase Realtime)

Phase 4: Testing + Hardening (Weeks 15–17)

- [ ] GitHub Actions CI pipeline
- [ ] Firebase Test Lab integration
- [ ] Maestro flows for critical paths
- [ ] Full E2E test suite including dual-scan inventory, WhatsApp Flows, CRM sync
- [ ] Manual UAT on device matrix
- [ ] Performance testing
- [ ] Report accuracy validation
- [ ] Backup restore drill

Phase 5: Launch (Weeks 17–20)

- [ ] Seed production reference data (products, stores, staff, shelves, holidays)
- [ ] Staff training (focus: new home screen, same POS flow, inventory count protocol, shift selection)
- [ ] Print and deploy shelf labels to first store
- [ ] Staged rollout: one store first, then expand
- [ ] Monitor for 1 week post-rollout
- [ ] Evaluate PITR upgrade based on transaction volume

Post-MVP

- [ ] Dynamic feature module split (on-demand delivery)
- [ ] Replace `:feature:pos-legacy` screens incrementally; extract Blink into `:core:payments`
- [ ] Spot check mode for inventory (single-scan, lightweight)
- [ ] Restaurant suite
- [ ] Advanced reporting and analytics (trend charts, forecasting)
- [ ] Full HR rules engine
- [ ] Marketing campaigns via WhatsApp
- [ ] Customer-facing loyalty web portal
- [ ] Paddle billing integration (if multi-brand SaaS)
- [ ] Driver route optimization, live GPS tracking, ETA
- [ ] Supplier management and purchase orders
- [ ] Cost allocation of operational supplies to stores
- [ ] AI chat: proactive alerts ("Heads up — Grand Baie is running low on receipt rolls")

---

22. Assumptions (updated)

- Supabase is the sole system of record. No data in Zoho Creator or CRM needs migration.
- Zoho CRM receives one-way pushes for support visibility only. CRM data is never read by the platform.
- Zoho Creator data is test/playground only and will be retired.
- The brand portal (loyalty.posterita.com) is a prototype that informed the design. It will be retired when the web console absorbs its functionality.
- Vercel hosts the web console. The loyalty Flask API on Vercel is retired.
- WhatsApp Flows require Meta approval which may take 1-3 weeks.
- Epson ePOS SDK supports shelf label printing (receipt-width barcode labels).
- A user works in exactly one store per day.
- Blink payment integration must be preserved.

- Mauritius DPA 2017 compliance required for customer data and consent.
- Inventory count dual-scan protocol applies to full counts. Spot checks (single scan, KPI-only) included in MVP.
- Customer-facing loyalty is WhatsApp-first during MVP. Web portal deferred.

---

23. Resolved Design Decisions (all concerns incorporated)

#	Question	Decision
1	POS codebase approach (C1)	Evaluate playground code for reuse. Architecture wins over preservation. Build `:feature:pos` fresh, adopt good patterns. No `:feature:pos-legacy` wrapper needed.
2	Barcode scanners (C2)	Both camera and Bluetooth HID supported. Configurable modes: auto-scan, scan-confirm, continuous batch.
3	Product catalogue (C3)	Reuse POS product selection screen for catalogue generation. Same grid, different output mode (PDF catalogue, barcode labels, shelf labels). `:feature:catalogue` module.
4	Customer volume (C4)	Expect ~10,000 customers/year. WhatsApp conversation costs manageable (~$50-100/mo at this volume).
5	Spot check (C5)	Yes, in MVP. Single-scan, no dual verification. Generates Shelf Accuracy KPI that indicates when full count is needed. Spot checks are reporting/early-warning, not authoritative inventory.
6	Barcode formats (C6)	Multiple supported: QR Code (recommended default), Code 128, Code 39, DataMatrix. Configurable per store.
7	Label printers (C7)	Zebra (enterprise labels), Epson (receipt labels), generic printers (PDF). `:core:printer` abstracts all three via `PrinterService` interface.
8	Points per rupee (C8)	Configurable per brand/organization via `organization_loyalty_config` table.
9	Multi-store	One store per user per day, no switching
10	Product catalog source	Supabase, created via backend API
11	Master data from Android	Thin client — direct API call, then pull caches
12	Data protection	DPA 2017, consent append-only, dual-scope, Meta-compliant
13	Loyalty source of truth	Supabase only. CRM is a push target for support.
14	Zoho Creator fate	Retired. All data modeled fresh in Supabase.
15	Inventory full count	Dual-scan mandatory on full counts. 3rd device tiebreak on dispute.
16	Shelf addressing	STORE-ZONE-SHELF-POSITION, QR Code default format
17	WhatsApp bot	Logic in backend. SalesIQ stays as middleware. Zobot → thin relay.
18	WhatsApp consent	WhatsApp Flows for registration (Meta approval needed), button fallback.
19	CRM role	One-way push target only, never read from. Support visibility only.
20	Reports	Backend-generated, served to web console, CSV/PDF export
21	Owner signup (v3.3)	Multi-step on Android + web. Single-field steps, keyboard never hides input. AI generates starter products. Business name not required.
22	Entity hierarchy (v3.3)	Owner → Account → Brand (1..n) → Store (1..n) → Terminal (1..n) → Device. Owner can have multiple brands.
23	AI product onboarding (v3.3)	AI generates 20-30 starter products during signup. Same enrichment pipeline as §29. Owner reviews 1-by-1 before master.
24	Owner vs staff login (v3.3)	Owner: PIN/biometric with 30-day refresh. Staff: staff picker on enrolled device + PIN with 7-day refresh.
25	In-app chat (v3.4)	AI assistant chat in MVP (highest ROI). Direct + group chat in Phase 3. AI uses same API endpoints as UI, scoped to user permissions.
26	Logistics (v3.4)	Template-based delivery workflow. Standard parcel (MVP), motorcycle handover + inter-store transfer (Phase 3). QR on every package.
27	QR deep routing (v3.4)	Every QR encodes a type field. App QR router parses payload → opens exact correct screen. Two classes: WhatsApp QR (customers) and internal QR (staff).
28	Operational supplies (v3.4)	Same `product` table, `product_class='operational'`. Never shown in POS. Separate stock alerts and warehouse dispatch.
29	Cash on delivery (v3.5)	COD tracked per shipment. Driver collects payment → records amount + evidence → deposits at store/office → manager reconciles. Cannot end shift with undeposited cash.
30	Label printer (v3.5)	QR label printer (Zebra or equivalent) is mandatory equipment per store. Fallback to receipt printer or PDF, but stores must budget for dedicated printer.
31	Android dynamic features (v3.6)	Play Feature Delivery (SplitInstallManager). Core always installed (~8MB). Feature modules download on first tap. Cashier never downloads logistics module.
32	Store cash collection	Daily sales cash: store manager declares → generates collection QR → driver scans + dual signature → transport → bank deposit with slip

	(v3.6)	photo → three-way reconciliation (declared vs collected vs deposited).
33	Barcode My Store (v3.6)	Guided shelf-by-shelf workflow: scan shelf → count distinct products → photo each → enter qty → AI identifies → owner reviews 1-by-1 → products created with auto-barcode → labels print in shelf walking order.
34	AI product ID from photos (v3.6)	Same Claude API pipeline as catalogue enrichment. Photo → AI suggests name/category/brand. Owner must accept/edit/skip. AI also flags potential duplicates across shelves.
35	Container/Import receiving (v3.7)	Full lifecycle: arrive → document vault → inspect per package → process products → release to store flexibly → cost allocation → PO reconciliation → close. Sell now, cost later.
36	Merchandiser role (v3.7)	New role: manages product pipeline from container to shelf. Has access to container receiving, inspection, cost allocation, product management, AI enrichment, stock release.
37	Claims workflow (v3.7)	Damage/shortage/wrong-item claims against supplier, freight, or insurance. Photo evidence from inspection. Tracked through resolution (credit note, replacement, payout, write-off).
38	Sell now, cost later (v3.7)	Products can be created with selling price and released to stores before container costing is complete. Cost price backfilled when overhead allocation runs. Margin reports flag "cost pending."

---

24. Backup and Recovery Strategy

Unchanged from v2 §24. Supabase Pro daily backups at launch, PITR when volume justifies. Backup checklist remains.

---

25. WhatsApp Customer Messaging

Architecture

```
Customer (WhatsApp)
    ↕
Meta Cloud API
    ↕
Zoho SalesIQ (WhatsApp middleware)
    ↕ webhook to backend
NestJS WhatsApp module
    ↕
Supabase (customer, wallet, consent)
    ↕ async BullMQ job
Zoho CRM (one-way push for support)
```

SalesIQ Zobot becomes a thin relay (~50 lines Deluge):

```
response = Map();
msg = message.get("text");
phone = visitor.get("phone");
wa_name = visitor.get("name");

// Forward everything to the backend
payload = Map();
payload.put("phone", phone);
payload.put("name", wa_name);
payload.put("message", msg);
backend_resp = invokeurl [ url: BACKEND_WHATSAPP_URL type: POST parameters: payload ];

// Return backend response to customer
response.put("action", "reply");
response.put("replies", backend_resp.get("replies"));
if(backend_resp.get("input") != null) { response.put("input", backend_resp.get("input")); }
return response;
```

WhatsApp Flows

Flow 1: Registration + Consent (QR scan in showroom)

Customer scans QR → opens WhatsApp with trigger → backend sends Flow: - Screen 1: Welcome + name/phone pre-filled - Screen 2: Optional email input - Screen 3: Consent checkboxes (promos + news, separately) - Screen 4: Confirmation with bonus points

Flow 2: Survey (earn bonus points) - Screen 1: 3 quick questions - Screen 2: Points awarded confirmation

Outbound Templates (require Meta approval)

Template	Trigger	Variables
`loyalty_welcome`	Registration	name, points
`points_earned`	POS purchase	name, points, store, balance
`voucher_issued`	Campaign/reward	name, desc, code, expiry
`voucher_expiring`	3 days before expiry (cron)	name, desc, code, days_left

`digital_receipt`	Sale + customer linked	name, order_ref, total, store
`survey_invite`	Campaign trigger	name, survey_name, points_reward
`consent_renewal`	Annual re-confirmation	name

CRM Connector Detail

Events that trigger a push to Zoho CRM:

Event	CRM Action	Retry Policy
Customer created/updated	Upsert Contact	5 retries, exponential backoff
Loyalty enrolled	Set Posterita_Loyalty_Enrolled	5 retries
Consent changed	Set WhatsApp_Marketing_Consent + timestamp + source	5 retries
Balance changed	Update Loyalty_Points field	5 retries
Voucher redeemed	Add CRM Note	3 retries

Dead letter queue after max retries. Alert to ops team. CRM is non-critical — platform operates normally if CRM is unreachable.

---

26. QR-First Customer Acquisition Funnel

Why This Matters

The QR code is the **single most important touchpoint** in the Retail OS. It's not a catalogue feature — it's the primary customer acquisition mechanism. Every QR scan that opens WhatsApp automatically captures the customer's phone number (WhatsApp requires it). That phone number becomes the universal customer identifier across POS, loyalty, WhatsApp, and CRM.

Without the QR→WhatsApp→phone pipeline, the system has no way to connect a physical showroom visitor to a digital customer record. With it, every product display, every printed catalogue, every receipt, and every storefront becomes a customer acquisition surface.

QR Touchpoint Map

Every QR code in the physical environment encodes a WhatsApp deep link that routes to the backend via the SalesIQ relay. The backend identifies the context from the message text and branches accordingly.

QR Location	Encoded URL	Trigger Message	Backend Action
Product label (on shelf, on product)	`https://wa.me/+230XXXXX?text=DETAILS%20YDMUSTART01`	`DETAILS YDMUSTART01`	Product lookup → send info card → check if enrolled → if not, prompt registration
Catalogue card (printed PDF)	`https://wa.me/+230XXXXX?text=DETAILS%20YDMUSTART01`	`DETAILS YDMUSTART01`	Same as product label
Storefront poster	`https://wa.me/+230XXXXX?text=WELCOME`	`WELCOME`	Welcome message → registration Flow → signup bonus
Receipt (printed after EVERY sale)	`https://wa.me/+230XXXXX?text=RECEIPT%20GBR-047`	`RECEIPT GBR-047`	Award points if enrolled → digital receipt → if not enrolled, prompt registration
Table tent / counter card	`https://wa.me/+230XXXXX?text=JOIN`	`JOIN`	Directly start registration Flow
Business card / flyer	`https://wa.me/+230XXXXX?text=HI`	`HI`	Welcome menu → store details, catalogue, loyalty
Social media / website	`https://wa.me/+230XXXXX?text=HI`	`HI`	Same as business card

The Acquisition Pipeline

Every QR scan enters the same pipeline, regardless of entry point:

```
CUSTOMER SCANS QR (any touchpoint)
    │
    ▼
WhatsApp opens → message auto-sent → SalesIQ relay → Backend
    │
    ▼
BACKEND: Extract phone number (always available from WhatsApp)
    │
    ▼
BACKEND: customer = lookup_by_phone(phone)
    │
    ├── FOUND (returning customer)
    │   │
    │   ▼
    │   Handle the trigger context:
    │   - DETAILS {sku} → send product card + "Welcome back, {name}! You have {pts} points."
    │   - RECEIPT {ref}  → send digital receipt + "You earned {pts} points on this purchase."
    │   - WELCOME / JOIN → "Welcome back! Your balance: {pts} points." + loyalty menu
    │   - HI             → main menu with personalized greeting
    │
    └── NOT FOUND (new customer)
        │
        ▼
        Start onboarding:
        1. Create customer record (phone + WhatsApp name)
        2. Send WhatsApp Flow: Registration + Consent
        3. On Flow completion:
           a. Update customer record (name, email from Flow)
           b. Record consent (promo/news, dual-scope)
           c. Create loyalty wallet
           d. Award signup bonus
           e. Push to CRM connector
        4. THEN handle the original trigger:
           - DETAILS {sku} → send product card (now personalized)
           - RECEIPT {ref}  → send digital receipt
           - etc.
```

Key insight: The customer gets what they asked for (product details, receipt, etc.) regardless of whether they complete registration. But the registration prompt is always presented first for new customers. If they dismiss the registration Flow, they still get the product info — but without loyalty benefits.

Phone Number as Universal Key

The phone number captured from WhatsApp becomes the **universal customer identifier**:

System	How Phone Is Used
POS	Cashier looks up customer by phone → links sale to customer → awards loyalty points
Loyalty wallet	Phone is the wallet lookup key (same as current Flask API: `GET /api/v1/balance/{phone}`)
WhatsApp	Phone is the conversation identifier — all inbound/outbound messages keyed to phone
Consent	Consent records tied to customer record (which is keyed by phone)
CRM	Customer synced to Zoho CRM Contact by phone (Mobile field)
Receipts	Digital receipt sent to phone via WhatsApp template
Vouchers	Vouchers assigned to customer (who is keyed by phone)
Campaigns	Audience targeting based on customer consent (keyed by phone)

The `customer` table has a unique constraint on `phone` (E.164 format). Phone normalization happens at the API layer — the same normalization from the Flask codebase (`strip spaces/dashes/parens, ensure starts with +`).

QR Code Generation

QR codes are generated at multiple points:

Generator	QR Content	Use Case
Backend API	`POST /v1/qr/product/{sku}` → returns QR image URL	Product labels, catalogue cards
Catalogue PDF generator	Auto-generated per product card in PDF	Printed catalogues
Receipt printer	ESC/POS QR command embedded in receipt	Post-sale enrollment
Web console	Bulk QR generation for shelf labels, product labels	Store setup
Shift attendance	QR stations for check-in/out (existing)	Staff ops

All product QR codes follow the same format: `https://wa.me/{whatsapp_number}?text={trigger}` . The WhatsApp number is configured per brand/store in the `organization` or `store` table.

Receipt QR — Mandatory On Every Receipt

Rule: Every POS receipt MUST print a QR code. No exceptions.

This is the highest-volume customer acquisition channel. Every transaction — whether the customer is enrolled or not, whether they paid cash or card, whether they bought one item or twenty — generates a receipt with a QR code. The receipt QR is the last touchpoint before the customer walks out the door.

Receipt QR format:

```
https://wa.me/+230XXXXX?text=RECEIPT%20{ORDER_REF}
```

What happens when customer scans the receipt QR:

```
CUSTOMER scans receipt QR
    │
    ▼
WhatsApp opens → sends "RECEIPT GBR-20260319-047"
    │
    ▼
BACKEND: lookup customer by phone
    │
    ├── ENROLLED CUSTOMER (linked to this order)
    │   │
    │   ▼
    │   "Thanks for your purchase at Grand Baie!
    │    You earned 31 points on this order.
    │    Your balance: 451 points.
    │
    │    [View Receipt] [My Points] [Vouchers]"
    │
    ├── ENROLLED CUSTOMER (not linked to this order)
    │   │
    │   ▼
    │   "Hi {name}! Looks like this purchase wasn't linked to your account.
    │    We've now added 31 points to your wallet.
    │    Balance: 451 points.
    │
    │    [View Receipt] [My Points]"
    │
    │   (Backend: retroactively link order to customer, award points)
    │
    └── NOT ENROLLED
        │
        ▼
        "Thanks for shopping at Grand Baie!
         Scan this receipt QR to earn loyalty points on every purchase.
         Join now and we'll add 31 points for this order + 100 bonus points!

         [Join Now] [Store Info] [No Thanks]"

        → [Join Now] triggers registration Flow
        → On completion: retroactively award points for this order + signup bonus
```

The receipt QR serves three purposes simultaneously:

1. **For enrolled customers who were linked at POS:** Confirmation of points earned + digital receipt + re-engagement touchpoint
2. **For enrolled customers who weren't linked at POS:** Retroactive point award — the customer gets their points even if the cashier forgot to scan their loyalty card. This is a recovery mechanism that increases loyalty satisfaction.
3. **For non-enrolled customers:** The single best acquisition moment. The customer just bought something, they're holding the receipt, and you're offering to give them points for this purchase right now. The conversion incentive is immediate and concrete ("31 points for this order + 100 bonus points").

Retroactive point award logic:

When a customer scans a receipt QR for an order they weren't linked to:

1. Backend verifies: order exists, order has no customer linked, order is less than 72 hours old
2. Backend links customer to order (update `order.customer_id`)
3. Backend awards points for that order (using the standard `loyalty_transaction` pipeline, idempotent on order UUID)
4. Backend sends confirmation with points earned and new balance
5. This is a **one-time action per order** — subsequent scans of the same receipt show the regular enrolled response

Receipt layout with QR:

```
┌──────────────────────────────┐
│     Funky Fish — Grand Baie  │
│     Receipt #GBR-20260319-047 │
│     19 Mar 2026 · 14:32      │
├──────────────────────────────┤
│ Reef Sandal Navy    x1  1,290.00 │
│ Canvas Tote Natural x2  1,300.00 │
│ Flip Flop Coral M   x1    490.00 │
├──────────────────────────────┤
│ Subtotal              3,080.00 │
│ VAT 15%                 462.00 │
│ TOTAL                 3,542.00 │
├──────────────────────────────┤
│ Paid: Cash            4,000.00 │
│ Change                  458.00 │
├──────────────────────────────┤
│                              │
│        ┌──────────┐          │
│        │ QR CODE  │          │
│        │          │          │
│        │  (large) │          │
│        │          │          │
│        └──────────┘          │
│                              │
│   Scan to earn loyalty points │
│   or get your digital receipt │
│                              │
│   Posterita Retail OS        │
└──────────────────────────────┘
```

QR size on receipt: Minimum 25mm × 25mm for reliable phone camera scanning. Epson ePOS SDK `addSymbol()` function with `QRCODE_MODEL_2`, error correction level M, module size 6+.

The receipt QR is printed EVEN IF the customer is already linked and points were awarded at POS. Reason: the receipt is a physical artifact that the customer takes home. It continues to be an entry point for WhatsApp engagement days later — "Hey, I got this receipt from you guys last week, let me check my points."

Receipt QR Fallback (no WhatsApp)

If a customer doesn't have WhatsApp or doesn't want to scan, the receipt still works as a traditional paper receipt. The QR is additive — it doesn't replace any receipt information. The text below the QR ("Scan to earn loyalty points or get your digital receipt") makes the value proposition clear without being pushy.

For stores where staff are trained to mention it: "If you scan the QR code at the bottom, you'll get your digital receipt and loyalty points on WhatsApp."

Showrooming Analytics

Because every QR scan routes through the backend, we get analytics for free:

Metric	How Measured
QR scans per day	Count of inbound WhatsApp messages matching trigger patterns
Scan-to-enrollment conversion	New customers created within 5 minutes of a product trigger
Most-scanned products	Rank products by `DETAILS {sku}` message frequency
Scan-to-purchase conversion	Customer scans product QR → purchases same product at POS within 24h
Entry point distribution	Which trigger messages are most common (DETAILS vs WELCOME vs JOIN vs RECEIPT vs HI)
Time-to-enrollment	Duration from first QR scan to completed registration Flow
Receipt QR scan rate	Receipts scanned vs total receipts printed — measures staff communication effectiveness
Receipt-to-enrollment conversion	Non-enrolled customers who scan receipt QR and complete registration
Retroactive point awards	Orders linked via receipt QR scan (customers who weren't identified at POS)

These feed into the Reports framework (§27) as a **Showroom Funnel** report.

New API Routes

- `POST /v1/qr/product/{sku}` — Generate product QR code (returns Cloudinary URL)
- `POST /v1/qr/store/{store_code}` — Generate store welcome QR
- `POST /v1/qr/receipt/{order_ref}` — Generate receipt QR
- `POST /v1/qr/bulk` — Generate QR codes for array of products/stores
- `GET /v1/analytics/showroom-funnel?store_id=&period=` — Showroom funnel report data

Schema Addition

```
-- Track every QR-originated interaction for showroom analytics
CREATE TABLE qr_scan_event (
    id              UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    phone           TEXT NOT NULL,              -- WhatsApp phone that scanned
    customer_id     UUID REFERENCES customer(id), -- NULL if new (not yet enrolled)
    trigger_type    TEXT NOT NULL,              -- 'product_details', 'welcome', 'join', 'receipt', 'hi'
    trigger_ref     TEXT,                       -- SKU for product, order_ref for receipt, NULL for general
    store_id        UUID REFERENCES store(id),  -- inferred from WhatsApp number routing
    is_new_customer BOOLEAN NOT NULL DEFAULT false,
    enrolled_at     TIMESTAMPTZ,               -- set when/if customer completes registration from this scan
    created_at      TIMESTAMPTZ NOT NULL DEFAULT now()
);
CREATE INDEX idx_qr_scan_phone ON qr_scan_event(phone);
CREATE INDEX idx_qr_scan_trigger ON qr_scan_event(trigger_type, trigger_ref);
CREATE INDEX idx_qr_scan_created ON qr_scan_event(created_at);
```

Implementation Priority

This is **Phase 2 critical path** — without it, there's no customer acquisition in the physical environment. The implementation order:

1. QR code generation endpoints (simple — encode URL as QR image via a library like `qrcode` on the backend)
2. WhatsApp inbound trigger routing (parse `DETAILS {sku}`, `WELCOME`, `JOIN`, `RECEIPT {ref}`, `HI`)
3. Customer lookup-or-create pipeline (phone normalization → lookup → create if needed → registration Flow)
4. Product info card response (image + description + action buttons)
5. QR scan event logging (for analytics)
6. Showroom funnel report

Everything downstream (catalogue PDFs with QR, receipt QR printing, storefront posters) uses the same QR generation endpoint. Build it once, use it everywhere.

---

27. Reports Framework

Architecture

Component	Technology
Query layer	SQL views + materialized views in Supabase
API	NestJS reports module with parameterized endpoints
Rendering	Web console (Recharts for charts, tables for data)
Export	Server-side PDF + CSV generation
Scheduling	Cron jobs for daily/weekly email summaries

MVP Reports (5)

1. **Daily Sales Summary** — revenue, order count, avg transaction, payment breakdown per store
2. **Till Discrepancy Summary** — discrepancy per cashier, resolution status, trend
3. **Device Fleet Health** — online/stale/offline, battery, app versions, sync performance
4. **Inventory Count Session Results** — matched vs disputed shelves, variance by product
5. **Enrollment Funnel** — new customers by channel, consent rate, points economy
6. **Showroom Funnel** — QR scans per day, scan-to-enrollment conversion, most-scanned products, scan-to-purchase conversion

Post-MVP Reports

POS: hourly heatmap, product ranking, refund report, discount usage, held order aging. Loyalty: top customers, voucher performance, consent breakdown. Workforce: attendance, leave balance, expense claims, task completion. Inventory: variance trend, shrinkage rate, stock alerts.

---

28. Product Display UI Improvements

View Mode Toggle

Add a toggle button in the top bar (list icon / grid icon): - **Compact list** (default): current 68px cards with 56px image strip, 2 columns — for speed - **Visual grid**: 120px cards with 80px square images, 2 columns — for browsing

Search Promotion

Move search to the top bar. Persistent search icon that expands to an inline search field on tap. When typing, product grid filters in real-time. When cleared, returns to category view.

Stock Indicator

Thin 3px colored bar at bottom of each product card: - Green (#2E7D32): stock ≥ 10 - Amber (#F57F17): stock 1-9 - Red (#E53935): stock 0 — card greyed, not tappable

Frequent Items

Auto-populated "★" category chip. Shows 12 most-sold products in last 7 days. Calculated from transaction data. First chip position (before ALL).

Category Color Accents

Each category gets a 4px left-border color accent on its chip. 6 predefined colors that cycle.

---

29. AI-Assisted Product Catalogue & Showrooming

The Problem

A product in the POS system starts as minimal data: name, SKU, price, category, one photo. That's enough for a cashier to ring it up, but it's not enough for:

- A **showroom catalogue PDF** that customers browse
- A **WhatsApp product card** sent when a customer scans a QR code
- A **web product page** with specs, features, and marketing copy
- A **barcode label** with a scannable description

Today, enriching product data is manual — someone has to write descriptions, pull specs from the manufacturer, take multiple photos, and format everything. For 50+ products across multiple brands, this doesn't scale.

The Solution: AI Enrichment with Human Gatekeeping

AI generates product content. Humans approve it field by field. Nothing touches the master record without explicit acceptance.

Product Data Lifecycle

```
┌──────────────┐    ┌──────────────────┐    ┌──────────────────┐    ┌──────────────┐
│ 1. INGEST    │ ──► │ 2. AI ENRICHMENT │ ──► │ 3. HUMAN REVIEW│ ──► │ 4. PUBLISH   │
│              │    │                  │    │                  │    │              │
│ Staff enters │    │ AI generates     │    │ Accept/Reject    │    │ Master record│
│ minimal data │    │ suggestions per  │    │ per field        │    │ updated      │
│              │    │ field            │    │ Edit if needed   │    │              │
│ Name         │    │ Short desc       │    │ Diff view        │    │ → POS card   │
│ SKU          │    │ Long desc        │    │ Batch mode       │    │ → Catalogue  │
│ Price        │    │ Features list    │    │ Confidence %     │    │ → WhatsApp   │
│ Category     │    │ Specs            │    │                  │    │ → Labels     │
│ 1 photo      │    │ Tags             │    │                  │    │ → Showroom   │
│              │    │ Marketing copy   │    │                  │    │              │
└──────────────┘    │ Catalogue blurb  │    └──────────────────┘    └──────────────┘
                    │ WhatsApp trigger  │
                    │ Suggested images  │
                    └──────────────────┘
```

Schema: Product Enrichment

```
-- ═══════════ PRODUCT (extended fields for catalogue) ═══════════
-- These columns are added to the existing product table
ALTER TABLE product ADD COLUMN short_description TEXT;          -- 1-2 lines, for POS card + WhatsApp
ALTER TABLE product ADD COLUMN long_description TEXT;           -- Full description for catalogue PDF
ALTER TABLE product ADD COLUMN features JSONB;                 -- ["Waterproof", "UV protection", ...]
ALTER TABLE product ADD COLUMN specs JSONB;                    -- {"material": "Neoprene", "weight": "220g", ...}
ALTER TABLE product ADD COLUMN tags TEXT[];                     -- searchable tags
ALTER TABLE product ADD COLUMN marketing_copy TEXT;             -- Catalogue blurb / sell sheet text
ALTER TABLE product ADD COLUMN whatsapp_trigger TEXT;           -- e.g. "DETAILS YDMUSTART01"
ALTER TABLE product ADD COLUMN catalogue_ready BOOLEAN NOT NULL DEFAULT false;
ALTER TABLE product ADD COLUMN enrichment_status TEXT NOT NULL DEFAULT 'pending'
    CHECK (enrichment_status IN ('pending', 'ai_generated', 'under_review', 'approved', 'needs_revision'));

-- ═══════════ AI ENRICHMENT SUGGESTIONS ═══════════
CREATE TABLE product_enrichment (
    id              UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    product_id      UUID NOT NULL REFERENCES product(id),
    field_name      TEXT NOT NULL,             -- 'short_description', 'long_description', 'features', etc.
    ai_suggestion   TEXT NOT NULL,             -- the AI-generated content
    confidence      NUMERIC(3,2),              -- 0.00 to 1.00 — how confident the AI is
    ai_model        TEXT NOT NULL DEFAULT 'claude-sonnet-4-20250514',
    ai_prompt_hash  TEXT,                      -- hash of the prompt used (for reproducibility)
    source_context  TEXT,                      -- what the AI was given (existing data, manufacturer URL, etc.)
    status          TEXT NOT NULL DEFAULT 'pending'
                    CHECK (status IN ('pending', 'accepted', 'rejected', 'edited')),
    reviewer_id     UUID REFERENCES "user"(id),
    reviewed_at     TIMESTAMPTZ,
    edited_value    TEXT,                      -- if status='edited', what the human changed it to
    created_at      TIMESTAMPTZ NOT NULL DEFAULT now()
);
CREATE INDEX idx_enrichment_product ON product_enrichment(product_id);
CREATE INDEX idx_enrichment_status ON product_enrichment(status);

-- ═══════════ PRODUCT MEDIA (multiple images per product) ═══════════
CREATE TABLE product_media (
    id              UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    product_id      UUID NOT NULL REFERENCES product(id),
    media_type      TEXT NOT NULL CHECK (media_type IN ('photo', 'video', 'document', 'ai_generated')),
    cloudinary_id   TEXT NOT NULL,
    url             TEXT NOT NULL,
    position        INTEGER NOT NULL DEFAULT 0,   -- display order
    is_primary      BOOLEAN NOT NULL DEFAULT false,
    is_catalogue    BOOLEAN NOT NULL DEFAULT false, -- included in catalogue PDF
    ai_alt_text     TEXT,                          -- AI-generated alt text for accessibility
    status          TEXT NOT NULL DEFAULT 'active'
                    CHECK (status IN ('active', 'archived', 'pending_review')),
    uploaded_by     UUID REFERENCES "user"(id),
    created_at      TIMESTAMPTZ NOT NULL DEFAULT now()
);
```

AI Enrichment Engine

The backend `catalogue` module calls the Anthropic API to generate product enrichment suggestions. The AI receives:

Input to AI (per product): - Product name, SKU, category, price - Primary photo (sent as base64 image to Claude) - Manufacturer name (if known) - Category context (what other products in same category look like) - Brand voice guidelines (tone, vocabulary, target audience) - Existing descriptions from similar products (few-shot examples)

AI generates (each as a separate `product_enrichment` row):

Field	What AI generates	Confidence signal
`short_description`	1-2 sentence summary for POS card and WhatsApp	High — product name + image usually sufficient
`long_description`	3-5 sentence description for catalogue	Medium — may need manufacturer specs
`features`	JSON array of 3-6 key features	Medium — inferred from image + category
`specs`	JSON object of technical specifications	Low — often needs manufacturer data
`tags`	Array of searchable tags	High — straightforward categorization
`marketing_copy`	2-3 sentences of sell-sheet prose	Medium — depends on brand voice accuracy
`whatsapp_trigger`	Command format: `DETAILS {SKU}`	High — formulaic
`ai_alt_text`	Accessibility alt text for primary image	High — image description

AI prompt structure:

```
You are a product copywriter for {brand_name}, a {brand_description} in Mauritius.

PRODUCT:
- Name: {name}
- Category: {category}
- Price: Rs {price}
- SKU: {sku}
[Image attached]

BRAND VOICE: {brand_voice_guidelines}

EXAMPLES of approved descriptions in this category:
{few_shot_examples}

Generate the following. Be accurate — do NOT invent specifications you can't see in the image.
For specs you're unsure about, set confidence to 0.0 and leave the field empty.

Return JSON:
{
  "short_description": { "value": "...", "confidence": 0.85 },
  "long_description": { "value": "...", "confidence": 0.70 },
  "features": { "value": ["...", "..."], "confidence": 0.60 },
  "specs": { "value": {"material": "...", "size": "..."}, "confidence": 0.40 },
  "tags": { "value": ["...", "..."], "confidence": 0.90 },
  "marketing_copy": { "value": "...", "confidence": 0.65 },
  "whatsapp_trigger": { "value": "DETAILS {sku}", "confidence": 0.99 }
}
```

Human Review Workflow

Web console: Product Enrichment Review screen

The review UI shows one product at a time, with each AI-suggested field as a reviewable card:

```
┌──────────────────────────────────────────────────────────┐
│  Reef Pro Sandal Navy  ·  YDMUSTART01  ·  Rs 1,290     │
│  [photo]                                                 │
│                                                          │
│  ┌─ Short Description ──────────── Confidence: 85% ──┐  │
│  │ AI suggests: "Durable reef sandal with quick-dry   │  │
│  │ navy straps, designed for beach-to-street wear."   │  │
│  │                                                    │  │
│  │ [✓ Accept]  [✎ Edit]  [✗ Reject]                   │  │
│  └────────────────────────────────────────────────────┘  │
│                                                          │
│  ┌─ Long Description ───────────── Confidence: 70% ──┐  │
│  │ AI suggests: "The Reef Pro Sandal Navy combines    │  │
│  │ rugged outdoor performance with everyday comfort.  │  │
│  │ Built with a contoured footbed and textured sole..."│  │
│  │                                                    │  │
│  │ [✓ Accept]  [✎ Edit]  [✗ Reject]                   │  │
│  └────────────────────────────────────────────────────┘  │
│                                                          │
│  ┌─ Specs ──────────────────────── Confidence: 40% ──┐  │
│  │ ⚠ LOW CONFIDENCE — AI may have guessed             │  │
│  │ AI suggests: {"material": "Synthetic/Rubber",      │  │
│  │               "sole": "Textured rubber"}           │  │
│  │                                                    │  │
│  │ [✓ Accept]  [✎ Edit]  [✗ Reject]                   │  │
│  └────────────────────────────────────────────────────┘  │
│                                                          │
│  [← Previous product]  [Skip]  [Next product →]          │
│  [Accept All High-Confidence (>80%)]                     │
└──────────────────────────────────────────────────────────┘
```

Review rules:

Rule	Enforcement
Nothing auto-accepted	All AI suggestions start as `pending`. Human must act.
Low confidence flagged	Fields with confidence < 50% show ⚠ warning and amber background
Batch accept available	"Accept All High-Confidence" button accepts only fields with confidence > 80%
Edit preserves AI trail	If human edits, both AI suggestion and human edit are stored
Reject leaves field empty	Rejected suggestions don't populate the master record
All fields must be reviewed	Product can't reach `catalogue_ready=true` until all enrichment rows are accepted/rejected/edited

Review states per product:

```
pending → ai_generated → under_review → approved (all fields reviewed)
                                      → needs_revision (some fields rejected, re-trigger AI with more context)
```

Batch Enrichment Workflow

For onboarding a new brand or catalogue refresh:

1. **Upload products CSV** — minimal data: name, SKU, price, category
2. **Upload product photos** — matched by SKU to Cloudinary

3. **Trigger batch enrichment** — backend queues AI enrichment for all pending products
4. **BullMQ job per product** — rate-limited to respect Anthropic API limits (e.g., 10 concurrent)
5. **Progress dashboard** — shows: total products, enriched, pending review, approved, rejected
6. **Review queue** — reviewer works through products one by one (or batch-accepts high-confidence)
7. **Catalogue generation** — once products reach `catalogue_ready=true`, they're available for PDF generation

Catalogue PDF Generation (ported from brand portal)

The brand portal already has a catalogue PDF studio with templates:

Template	Layout	Use Case
Credit card	Small product card with image, price, QR	Pocket reference cards
A5 portrait	Half-page product card	Showroom sell sheets
Compact sheet	Multiple products per page	Full catalogue booklet
Flyer	Single product, large image	Window display / promo

Each template includes: - Product image from Cloudinary - Name, price, short description - WhatsApp QR code (links to product trigger: `DETAILS {SKU}`) - Product barcode (for POS scanning from the catalogue itself) - Brand logo and footer

PDF generation endpoint: `POST /v1/catalogue/generate` - Input: product IDs + template + options (show price, show stock, headline, footer) - Output: PDF file uploaded to Cloudinary, URL returned - Uses: Puppeteer or @react-pdf/renderer on the backend

Showrooming Flow (complete customer journey)

```
1. CUSTOMER walks into showroom
2. CUSTOMER sees product on display
3. CUSTOMER scans QR code on product label / catalogue card
   → QR encodes: https://wa.me/+230XXXXX?text=DETAILS%20YDMUSTART01
4. WhatsApp opens with pre-filled message "DETAILS YDMUSTART01"
5. SalesIQ relay → backend WhatsApp module
6. Backend looks up product by trigger code
7. Backend sends WhatsApp reply:
   - Product image (from Cloudinary)
   - Short description (AI-generated, human-approved)
   - Price
   - "Want to know more?" buttons: [Full Specs] [Book Test Ride] [Find Store]
8. CUSTOMER taps [Full Specs] → receives long description + specs
9. CUSTOMER taps [Book Test Ride] → WhatsApp Flow for appointment
10. OR: CUSTOMER takes catalogue card to cashier → cashier scans barcode → POS
```

AI Safety Controls

Control	Implementation
AI never writes to master record directly	All suggestions go to `product_enrichment` table, not `product`
Human review required for every field	`catalogue_ready` flag only set when all fields reviewed
Confidence transparency	AI self-reports confidence; low-confidence fields flagged visually
Audit trail	Every suggestion, acceptance, rejection, and edit is logged with user + timestamp
Prompt versioning	`ai_prompt_hash` tracks which prompt version generated each suggestion
Rate limiting	Batch enrichment capped at 10 concurrent API calls to control costs
Cost visibility	Dashboard shows: API calls this month, estimated cost, cost per product
No hallucinated specs	Prompt explicitly instructs AI: "Do NOT invent specifications you can't verify from the image. Set confidence to 0 for unknown specs."
Re-enrichment	If product data changes (new photo, category change), AI can re-run. Previous suggestions archived, not deleted.

API Routes (new)

- `POST /v1/catalogue/enrich/{product_id}` — Trigger AI enrichment for one product
- `POST /v1/catalogue/enrich/batch` — Trigger enrichment for array of product IDs
- `GET /v1/catalogue/enrichment-queue` — Review queue (products with pending suggestions)
- `GET /v1/catalogue/enrichment/{product_id}` — All suggestions for a product
- `POST /v1/catalogue/enrichment/{id}/accept` — Accept a suggestion
- `POST /v1/catalogue/enrichment/{id}/reject` — Reject a suggestion
- `POST /v1/catalogue/enrichment/{id}/edit` — Accept with edits
- `POST /v1/catalogue/enrich/{product_id}/accept-high-confidence` — Batch accept >80% confidence
- `POST /v1/catalogue/generate` — Generate PDF catalogue from approved products
- `GET /v1/catalogue/templates` — List available PDF templates

- `GET /v1/catalogue/stats` — Enrichment progress dashboard data

Cost Estimate

Anthropic API pricing for product enrichment: - ~1,500 input tokens per product (name + SKU + category + prompt + image) - ~500 output tokens per product (JSON response) - At Claude Sonnet pricing (~$3/M input, $15/M output): - **Per product:** ~$0.005 input + $0.0075 output ≈ **$0.01 per product** - **100 products:** ~$1.25 - **1,000 products:** ~$12.50

Re-enrichment (e.g., after photo update) is the same cost per product. Extremely affordable.

---

30. In-App Chat & AI Assistant

Why Chat Is Core

Chat is not a "nice to have" social feature — it's the operational nervous system. A supervisor needs to tell a cashier to restock aisle 3. A driver needs to confirm a delivery address. An owner needs to ask "what were sales at Grand Baie yesterday?" and get an answer without navigating dashboards.

Three Chat Modes

1. Direct Messages (user → user) - Any staff member can message any other within the same brand - Messages are persistent, searchable, timestamped - Supports text, photos (from `:core:media`), voice notes, and location sharing - Unread badge on the chat tab in the bottom nav - Notifications via push (FCM)

2. Group Chats - Created by supervisors or admins - Predefined group types: Store team, Brand managers, Drivers, All staff - Custom groups allowed - Same message types as direct messages - @mention for targeted notifications

3. AI Assistant Chat - Every user has access to an AI chat thread - The AI can query the system on the user's behalf (scoped to their permissions) - The AI can execute actions (with the same capability/approval gates from §18)

AI Assistant Capabilities

The AI assistant is a conversational interface to the entire Retail OS:

User says	AI does	Permission required
"What were sales at Grand Baie yesterday?"	Queries daily sales report, returns summary	View reports
"How many sandals do we have in stock?"	Queries product stock, returns count	View products
"Create a leave request for Friday"	Creates leave request in the system	Submit requests
"Who's working the morning shift tomorrow?"	Queries shift schedule	View shifts
"Assign restocking task to Ravi"	Creates task, assigns to Ravi	Create tasks (supervisor+)
"What's Marie Laurent's loyalty balance?"	Queries customer wallet	View customers
"Send the catalogue PDF to this WhatsApp number"	Triggers WhatsApp template	Send messages
"Generate an inventory count report for last week"	Generates report, returns download link	View reports
"Show me unresolved discrepancies"	Queries reconciliation data	View reconciliation
"Approve Amina's leave request"	Executes approval	Approve requests (supervisor+)

Safety: The AI assistant is bound by the same capability model as the user. A cashier asking "approve this leave request" gets "You don't have permission to approve requests." Actions that require manager approval (from §18 AI safety gates) still require manager approval even when triggered via chat.

Implementation: The AI assistant calls the same backend API endpoints that the UI uses. No special backdoor. The chat module sends the user's message + their JWT to a backend `/v1/chat/ai` endpoint. The backend uses the Anthropic API with tool use — the tools are the existing API endpoints. The AI's JWT is the user's JWT, so all permission checks apply.

Schema: Chat

```
CREATE TABLE chat_thread (
    id              UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    thread_type     TEXT NOT NULL CHECK (thread_type IN ('direct', 'group', 'ai')),
    name            TEXT,                       -- NULL for direct, group name for groups, "AI Assistant" for ai
    organization_id UUID NOT NULL REFERENCES organization(id),
    created_by      UUID NOT NULL REFERENCES "user"(id),
    created_at      TIMESTAMPTZ NOT NULL DEFAULT now(),
    updated_at      TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE TABLE chat_thread_member (
    id              UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    thread_id       UUID NOT NULL REFERENCES chat_thread(id),
    user_id         UUID NOT NULL REFERENCES "user"(id),
    role            TEXT NOT NULL DEFAULT 'member'
                    CHECK (role IN ('member', 'admin')),
    last_read_at    TIMESTAMPTZ,
    joined_at       TIMESTAMPTZ NOT NULL DEFAULT now(),
    UNIQUE(thread_id, user_id)
);

CREATE TABLE chat_message (
    id              UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    thread_id       UUID NOT NULL REFERENCES chat_thread(id),
    sender_id       UUID REFERENCES "user"(id),  -- NULL for system/AI messages
    message_type    TEXT NOT NULL DEFAULT 'text'
                    CHECK (message_type IN ('text', 'photo', 'voice', 'location', 'system', 'ai_response', 'ai_action')),
    content         TEXT,
    media_url       TEXT,                       -- Cloudinary URL for photos/voice
    metadata        JSONB,                      -- location coords, AI action details, etc.
    created_at      TIMESTAMPTZ NOT NULL DEFAULT now()
);
CREATE INDEX idx_chat_msg_thread ON chat_message(thread_id, created_at);
```

Android Module

```
:feature:chat                ← Direct messages, groups, AI assistant, unread badges
```

Added to the bottom nav as a 5th tab: Home / POS / Chat / Tasks / More

⚠ CONCERN: Chat scope for MVP

Full chat (direct + group + AI) is a significant feature. Recommendation:

- **MVP:** AI assistant chat only (highest ROI — every user gets a query/action interface)
- **Phase 3:** Direct messages and group chat (lower priority, WhatsApp already covers urgent staff communication)

---

31. Logistics & Delivery

The Problem

Goods move between warehouses, stores, and customers. A driver picks up packages, delivers them, collects signatures, and reports back. Different delivery types have radically different workflows — delivering a carton of sandals is simple, but delivering a motorcycle to a customer involves key handover, photo documentation, testing checklist, and warranty explanation.

Entity Model

```
Shipment (the overall delivery job)
  └── ShipmentPackage (1..n packages in the shipment)
        └── ShipmentPackageLine (1..n items in each package)

DeliveryTemplate (defines the workflow for a delivery type)
  └── DeliveryTemplateStep (ordered steps in the template)
```

Schema: Logistics

```
-- ═══════════ DELIVERY TEMPLATE ═══════════
CREATE TABLE delivery_template (
    id              UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    organization_id UUID NOT NULL REFERENCES organization(id),
    name            TEXT NOT NULL,               -- "Standard Parcel", "Motorcycle Handover", "Inter-Store Transfer"
    code            TEXT NOT NULL UNIQUE,         -- "STD_PARCEL", "MOTO_HANDOVER", "STORE_TRANSFER"
    description     TEXT,
    is_default      BOOLEAN NOT NULL DEFAULT false,
    signature_required BOOLEAN NOT NULL DEFAULT true,
    photo_required  BOOLEAN NOT NULL DEFAULT false,
    active          BOOLEAN NOT NULL DEFAULT true,
    created_at      TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE TABLE delivery_template_step (
    id              UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    template_id     UUID NOT NULL REFERENCES delivery_template(id),
    step_number     INTEGER NOT NULL,
    step_type       TEXT NOT NULL
                    CHECK (step_type IN (
```

```
                        'scan_package',     -- scan QR to confirm package
                        'photo_capture',    -- take photo (before/after/damage)
                        'checklist',        -- tick off checklist items
                        'signature',        -- capture signature on phone
                        'text_input',       -- free text (notes, serial numbers)
                        'key_handover',     -- record key serial/count
                        'warranty_explain', -- confirm warranty was explained
                        'collect_payment',  -- COD: collect cash/card, record amount, take evidence photo
                        'qr_generate',      -- generate QR for recipient
                        'confirmation'      -- final confirmation
                    )),
    title           TEXT NOT NULL,           -- "Scan all packages", "Take delivery photo"
    description     TEXT,                    -- Instructions for the driver
    is_required     BOOLEAN NOT NULL DEFAULT true,
    checklist_items JSONB,                   -- for checklist type: ["Brakes tested", "Lights working", ...]
    created_at      TIMESTAMPTZ NOT NULL DEFAULT now(),
    UNIQUE(template_id, step_number)
);

-- ========== SHIPMENT ==========
CREATE TABLE shipment (
    id              UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    organization_id UUID NOT NULL REFERENCES organization(id),
    template_id     UUID NOT NULL REFERENCES delivery_template(id),
    shipment_code   TEXT NOT NULL UNIQUE,        -- "SHP-20260319-001"
    shipment_type   TEXT NOT NULL
                    CHECK (shipment_type IN (
                        'warehouse_to_store',
                        'store_to_store',
                        'store_to_customer',
                        'warehouse_to_customer',
                        'pickup_request'
                    )),
    status          TEXT NOT NULL DEFAULT 'created'
                    CHECK (status IN ('created', 'assigned', 'in_transit', 'at_destination',
                                      'delivering', 'completed', 'failed', 'cancelled')),
    origin_store_id     UUID REFERENCES store(id),
    origin_warehouse_id UUID,                    -- future: warehouse entity
    destination_store_id UUID REFERENCES store(id),
    destination_customer_id UUID REFERENCES customer(id),
    destination_address TEXT,
    driver_user_id  UUID REFERENCES "user"(id),
    requested_by    UUID REFERENCES "user"(id),
    requested_at    TIMESTAMPTZ NOT NULL DEFAULT now(),
    assigned_at     TIMESTAMPTZ,
    picked_up_at    TIMESTAMPTZ,
    delivered_at    TIMESTAMPTZ,
    -- Cash on delivery
    cod_required    BOOLEAN NOT NULL DEFAULT false,
    cod_amount      NUMERIC(12,2) DEFAULT 0,       -- amount to collect from recipient
    cod_currency    TEXT DEFAULT 'MUR',
    cod_collected   BOOLEAN NOT NULL DEFAULT false,
    cod_collected_at TIMESTAMPTZ,
    cod_collected_by UUID REFERENCES "user"(id),   -- driver who collected
    cod_reconciled  BOOLEAN NOT NULL DEFAULT false, -- matched to till/deposit
    cod_reconciled_at TIMESTAMPTZ,
    notes           TEXT,
    created_at      TIMESTAMPTZ NOT NULL DEFAULT now(),
    updated_at      TIMESTAMPTZ NOT NULL DEFAULT now()
);

-- ========== COD PAYMENT (cash collected by driver) ==========
CREATE TABLE cod_payment (
    id              UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    shipment_id     UUID NOT NULL REFERENCES shipment(id),
    driver_user_id  UUID NOT NULL REFERENCES "user"(id),
    amount          NUMERIC(12,2) NOT NULL,
    currency        TEXT NOT NULL DEFAULT 'MUR',
    payment_method  TEXT NOT NULL DEFAULT 'cash'
                    CHECK (payment_method IN ('cash', 'card', 'mobile_money')),
    collected_at    TIMESTAMPTZ NOT NULL DEFAULT now(),
    -- Reconciliation: driver must hand cash to store/office
    deposited       BOOLEAN NOT NULL DEFAULT false,
    deposited_at    TIMESTAMPTZ,
    deposited_to    TEXT,                        -- "Grand Baie till", "Head office", etc.
    deposit_evidence_url TEXT,                   -- photo of cash count / deposit slip
    reconciled      BOOLEAN NOT NULL DEFAULT false,
    reconciled_by   UUID REFERENCES "user"(id),
    reconciled_at   TIMESTAMPTZ,
    notes           TEXT,
    idempotency_key UUID UNIQUE,
    created_at      TIMESTAMPTZ NOT NULL DEFAULT now()
);

-- ========== SHIPMENT PACKAGE ==========
CREATE TABLE shipment_package (
    id              UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    shipment_id     UUID NOT NULL REFERENCES shipment(id),
    package_code    TEXT NOT NULL UNIQUE,         -- "PKG-20260319-001-01"
    barcode_data    TEXT NOT NULL UNIQUE,         -- QR payload for scanning
    package_type    TEXT NOT NULL DEFAULT 'carton'
                    CHECK (package_type IN ('carton', 'pallet', 'envelope', 'vehicle', 'loose', 'other')),
    description     TEXT,
    weight_kg       NUMERIC(8,2),
    status          TEXT NOT NULL DEFAULT 'created'
                    CHECK (status IN ('created', 'labelled', 'picked_up', 'in_transit', 'delivered', 'received', 'damaged')),
    label_printed   BOOLEAN NOT NULL DEFAULT false,
    picked_up_at    TIMESTAMPTZ,
    delivered_at    TIMESTAMPTZ,
    received_at     TIMESTAMPTZ,                 -- when recipient scans to confirm receipt
    received_by     UUID REFERENCES "user"(id),
    created_at      TIMESTAMPTZ NOT NULL DEFAULT now()
);

-- ========== PACKAGE LINE (items in package) ==========
CREATE TABLE shipment_package_line (
```

```
    id              UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    package_id      UUID NOT NULL REFERENCES shipment_package(id),
    product_id      UUID REFERENCES product(id),            -- NULL for non-catalogue items
    supply_item_id  UUID REFERENCES operational_supply(id), -- NULL for catalogue items
    description     TEXT NOT NULL,               -- always set (product name or free text)
    quantity        INTEGER NOT NULL DEFAULT 1,
    serial_number   TEXT,                        -- for vehicles, electronics
    notes           TEXT,
    created_at      TIMESTAMPTZ NOT NULL DEFAULT now()
);

-- ========== DELIVERY STEP COMPLETION ==========
CREATE TABLE shipment_step_completion (
    id              UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    shipment_id     UUID NOT NULL REFERENCES shipment(id),
    template_step_id UUID NOT NULL REFERENCES delivery_template_step(id),
    completed_by    UUID NOT NULL REFERENCES "user"(id),
    status          TEXT NOT NULL DEFAULT 'completed'
                    CHECK (status IN ('completed', 'skipped', 'failed')),
    data            JSONB,                       -- checklist answers, text input, key serial, etc.
    photo_url       TEXT,                        -- Cloudinary URL if photo step
    signature_url   TEXT,                        -- Cloudinary URL if signature step
    completed_at    TIMESTAMPTZ NOT NULL DEFAULT now()
);
```

Default Delivery Templates

The system ships with these built-in templates (editable by admin):

Template 1: Standard Parcel | Step | Type | Required | |---|---|---| | 1 | `scan_package` — Scan each package QR | Yes | | 2 | `photo_capture` — Photo of packages at pickup | No | | 3 | `photo_capture` — Photo at delivery location | Yes | | 4 | `collect_payment` — If COD: collect cash, enter amount, photo of cash/receipt | **If COD** | | 5 | `signature` — Recipient signs on phone | Yes | | 6 | `confirmation` — Confirm delivery complete | Yes |

Template 2: Motorcycle Handover | Step | Type | Required | |---|---|---| | 1 | `scan_package` — Scan vehicle package QR | Yes | | 2 | `photo_capture` — Photo of vehicle condition (4 angles) | Yes | | 3 | `text_input` — Record VIN / chassis number | Yes | | 4 | `key_handover` — Record key set count + serial | Yes | | 5 | `checklist` — Test checklist: brakes, lights, horn, battery charge, tires, mirrors | Yes | | 6 | `warranty_explain` — Confirm warranty terms explained | Yes | | 7 | `text_input` — Customer questions / notes | No | | 8 | `collect_payment` — If COD: collect balance due, enter amount, photo evidence | **If COD** | | 9 | `signature` — Customer signs acceptance | Yes | | 10 | `photo_capture` — Customer with vehicle | No | | 11 | `confirmation` — Handover complete | Yes |

Template 3: Inter-Store Transfer | Step | Type | Required | |---|---|---| | 1 | `scan_package` — Scan packages at origin store | Yes | | 2 | `scan_package` — Scan packages at destination store | Yes | | 3 | `signature` — Receiving store staff signs | Yes | | 4 | `confirmation` — Transfer complete | Yes |

Template 4: Pickup Request | Step | Type | Required | |---|---|---| | 1 | `confirmation` — Driver confirms pickup location | Yes | | 2 | `scan_package` — Scan packages at pickup | Yes | | 3 | `photo_capture` — Photo of loaded packages | No | | 4 | `confirmation` — Pickup complete, in transit | Yes |

Package Labels

Every package gets a QR label printed before shipment:

QR encoding: `{"t":"pkg","id":"PKG-20260319-001-01","s":"SHP-20260319-001"}`

When scanned, the QR deep-routes to: the package detail screen showing shipment context, contents, and current status. If the scanner is the driver, it opens the delivery workflow. If the scanner is the receiving store, it opens the receipt confirmation form.

Driver Flow (Android)

```
DRIVER opens app → sees assigned shipments
  │
  ├── Tap shipment → see packages + destination + template
  │
  ├── At origin: scan each package QR → confirms pickup → status: in_transit
  │
  ├── At destination: template steps execute in order
  │   Step 1: Scan packages (verify all present)
  │   Step 2: Photo (per template)
  │   Step 3: Checklist (per template)
  │   Step 4: Signature (recipient signs on phone screen)
  │   Step 5: Confirmation
  │
  └── Shipment complete → all data synced to server
```

Store-to-Store Request Flow

```
STORE A needs to send product to STORE B:
  1. Store A staff creates shipment request (selects products, destination store)
  2. If products are from own inventory → auto-approved
  3. If products requested from warehouse → needs approval
  4. Logistics assigns driver
  5. Driver picks up → delivers → Store B scans to receive
  6. Inventory automatically adjusts (Store A -, Store B +)
```

Cash on Delivery (COD)

Drivers collect cash (or card) payments at delivery. This cash must be tracked, deposited, and reconciled — it's real money in a driver's pocket until it reaches the till or office.

COD Collection Flow:

```
SHIPMENT marked as COD (cod_required=true, cod_amount=Rs 3,542)
    │
    ▼
DRIVER arrives at destination, executes delivery template
    │
    ▼
COLLECT PAYMENT step appears (shows amount due: Rs 3,542)
    │
    ├── Driver enters amount received: [Rs 4,000  ]
    │   Payment method: [Cash ▾]
    │   [📷 Take photo of cash / receipt]
    │   Change given: Rs 458 (auto-calculated)
    │
    ▼
COD payment record created (synced to server)
    │
    ▼
DRIVER continues to signature + confirmation steps
    │
    ▼
End of shift: DRIVER RECONCILIATION
    │
    ▼
Driver sees: "You collected Rs 12,400 across 4 deliveries today"
    │
    ├── Hand cash to store till → store manager confirms → photo of count
    │   OR
    ├── Deposit at office → supervisor confirms → photo of deposit slip
    │
    ▼
cod_payment.deposited = true, deposit_evidence_url stored
    │
    ▼
MANAGER reviews in web console → marks reconciled
```

Driver COD dashboard (Android): - Shows: total cash collected today, breakdown per delivery, deposit status - Cannot complete shift without depositing all collected cash - Alert to supervisor if driver has undeposited cash > 24 hours old

COD Reports (added to §27): - Driver cash collection summary (per driver, per day) - Undeposited cash aging (which drivers are holding cash) - COD vs prepaid delivery breakdown - Collection discrepancies (amount collected ≠ amount due)

Store Cash Collection (Daily Sales Cash → Bank)

Separate from COD. This is the **daily operational cash** sitting in store tills from sales. A driver physically picks it up and transports it to the bank or head office. This is a major cash-handling process that needs tight tracking.

The flow:

```
END OF DAY: Store closes till → till reconciliation completed (expected vs counted)
    │
    ▼
CASH READY FOR COLLECTION
    Till balance: Rs 47,200 (after float retained)
    Store manager seals cash in security bag
    Manager enters amount in app → takes photo of sealed bag
    App generates a COLLECTION QR for this cash bag
    │
    ▼
DRIVER ARRIVES AT STORE
    Driver scans store's Collection QR
    → App opens: "Cash collection at Grand Baie: Rs 47,200"
    → Driver verifies amount with store manager
    → Both sign on phone (dual signature)
    → Photo of sealed bag with both present
    │
    ▼
DRIVER IN TRANSIT (may visit multiple stores)
    Driver dashboard shows:
    - Grand Baie: Rs 47,200 ✓ collected
    - Port Louis: Rs 31,800 ✓ collected
    - Tribeca Mall: pending
    Total cash in vehicle: Rs 79,000
    │
    ▼
DRIVER AT BANK / HEAD OFFICE
    Driver deposits cash
    → Enters deposit slip number
    → Takes photo of deposit slip / bank receipt
    → Per-store breakdown recorded
    │
    ▼
RECONCILIATION
    Manager reviews in web console:
    - Store reported: Rs 47,200
    - Driver collected: Rs 47,200 (signed by both)
    - Bank deposited: Rs 47,200 (deposit slip attached)
    → Marks reconciled ✓

    ANY DISCREPANCY → flagged, requires investigation
```

Schema:

```
CREATE TABLE cash_collection (
    id              UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    organization_id UUID NOT NULL REFERENCES organization(id),
    store_id        UUID NOT NULL REFERENCES store(id),
    till_session_id UUID REFERENCES till_session(id),     -- linked to the closed till
    collection_date DATE NOT NULL,
    -- Store side
    amount_declared NUMERIC(12,2) NOT NULL,               -- what store says is in the bag
    currency        TEXT NOT NULL DEFAULT 'MUR',
    bag_seal_number TEXT,                                   -- security bag serial
    declared_by     UUID NOT NULL REFERENCES "user"(id),  -- store manager
    declared_at     TIMESTAMPTZ NOT NULL,
    bag_photo_url   TEXT,                                   -- photo of sealed bag
    collection_qr   TEXT NOT NULL UNIQUE,                   -- QR code for driver to scan
    -- Driver side
    status          TEXT NOT NULL DEFAULT 'pending'
                    CHECK (status IN ('pending', 'collected', 'in_transit', 'deposited', 'reconciled', 'discrepancy')),
    collected_by    UUID REFERENCES "user"(id),            -- driver
    collected_at    TIMESTAMPTZ,
    driver_confirmed_amount NUMERIC(12,2),                -- what driver confirms
    collection_signature_store TEXT,                        -- signature URL (store manager)
    collection_signature_driver TEXT,                       -- signature URL (driver)
    collection_photo_url TEXT,                              -- photo at collection
    -- Bank deposit
    deposited_at    TIMESTAMPTZ,
    deposit_slip_number TEXT,
    deposit_slip_photo_url TEXT,
    deposit_amount  NUMERIC(12,2),                          -- actual amount deposited
    deposit_bank    TEXT,                                   -- bank name
    -- Reconciliation
    reconciled      BOOLEAN NOT NULL DEFAULT false,
    reconciled_by   UUID REFERENCES "user"(id),
    reconciled_at   TIMESTAMPTZ,
    discrepancy_amount NUMERIC(12,2) DEFAULT 0,
    discrepancy_notes TEXT,
    created_at      TIMESTAMPTZ NOT NULL DEFAULT now(),
    updated_at      TIMESTAMPTZ NOT NULL DEFAULT now()
);
```

Cash Collection Reports (added to §27): - Daily cash collection summary (per store, per driver) - Cash in transit (what's currently being transported, by whom) - Deposit reconciliation (declared vs collected vs deposited — three-way match) - Discrepancy aging (unresolved cash discrepancies) - Float management (how much cash each store retains overnight)

API Routes (added to §12): - `POST /v1/cash-collection/declare` — Store declares cash ready for collection - `POST /v1/cash-collection/{id}/collect` — Driver scans QR, confirms collection, dual signature - `POST /v1/cash-collection/{id}/deposit` — Driver records bank deposit - `POST /v1/cash-collection/{id}/reconcile` — Manager reconciles - `GET /v1/cash-collection/in-transit` — Cash currently being transported - `GET /v1/cash-collection/report?period=` — Collection reconciliation report

Mandatory QR Label Printer Per Store

Every store MUST have a QR label printer. This is non-negotiable infrastructure.

The label printer is the most-used hardware in the Retail OS after the POS terminal itself. It prints:

What	When	Volume
Product barcode labels	New stock arrives, price changes, damaged labels	Daily
Shelf labels	Store setup, reorganization, damaged labels	Weekly
Package labels	Every outgoing shipment	Per shipment
Receipt QR (if no receipt printer)	Backup for QR printing	Rare
Inventory count zone cards	Before full count sessions	Per count
Asset tags	New asset acceptance	As needed
Return labels	Customer returns processing	As needed

Store setup checklist (updated): - [ ] POS terminal (Android device) - [ ] Receipt printer (Epson ePOS, Bluetooth/WiFi/USB) - [ ] **QR label printer (Zebra or equivalent, Bluetooth/USB) — MANDATORY** - [ ] Barcode scanner (camera or external Bluetooth) - [ ] Cash drawer - [ ] Attendance QR stations (printed, posted at entrance)

If a store doesn't have a label printer, the system falls back to: 1. Printing labels on the receipt printer (lower quality, not adhesive, but functional) 2. Generating a PDF of labels → print on any office printer → cut and tape

But the fallback is for emergencies. Every store should budget for a Zebra ZD220 or equivalent (~$200-300).

Android Module

```
:feature:logistics           ← Driver shipments, package scanning, delivery template execution, COD collection, labels
```

⚠ CONCERN: Logistics is a major module

This is essentially a mini-TMS (Transport Management System). For MVP, recommend:

- **MVP:** Package labels + scanning + basic delivery confirmation (Template 1: Standard Parcel)

- **Phase 3:** Full template system, motorcycle handover, inter-store transfers
- **Post-MVP:** Driver route optimization, live tracking, ETA

---

32. Universal QR Deep Routing

The Principle

Every QR code in the Posterita ecosystem is not just an identifier — it's a deep link that opens exactly the right screen in the app for the person scanning it.

The QR payload always contains a type field that tells the app what to do:

QR Payload Registry

QR Type	Payload	What it opens	Who scans it
`shelf`	`{"t":"shelf","s":"GB","z":"003","n":"012","p":"B"}`	Inventory count: open shelf for counting	Staff during inventory
`pkg`	`{"t":"pkg","id":"PKG-001","s":"SHP-001"}`	Package detail → delivery form (driver) or receipt form (store)	Driver, receiving staff
`product`	`{"t":"product","sku":"YDMUSTART01"}`	Product detail screen (POS) or catalogue view	Staff, customers via catalogue
`receipt`	`wa.me/{num}?text=RECEIPT%20{ref}`	WhatsApp → loyalty points + digital receipt	Customer
`attendance_in`	`{"t":"att","dir":"in","store":"GB","hash":"..."}`	Clock-in form → auto-submit with timestamp	Staff at shift start
`attendance_out`	`{"t":"att","dir":"out","store":"GB","hash":"..."}`	Clock-out form → auto-submit with timestamp	Staff at shift end
`customer_reg`	`wa.me/{num}?text=JOIN`	WhatsApp → registration Flow	Customer in store
`device_enroll`	`{"t":"enroll","token":"...","store":"..."}`	Device enrollment flow	Admin setting up device
`shipment`	`{"t":"ship","id":"SHP-001"}`	Shipment overview (driver: delivery steps, store: receipt)	Driver, receiving staff
`task`	`{"t":"task","id":"TSK-001"}`	Task detail screen	Staff assigned to task
`approval`	`{"t":"approval","id":"APR-001"}`	Approval form (approve/reject)	Supervisor

How It Works in the App

`:core:scanner` emits a `ScanResult(raw_data, format)`. The app's QR router parses the payload:

```
// QR Deep Router (in :core:navigation)
fun routeQR(scanResult: ScanResult): NavigationTarget {
    val payload = parseQRPayload(scanResult.data)

    return when (payload.type) {
        "shelf"    -> NavigationTarget.InventoryShelfCount(payload.shelfData)
        "pkg"      -> NavigationTarget.PackageDetail(payload.packageId, userRole)
        "product"  -> NavigationTarget.ProductDetail(payload.sku)
        "att"      -> NavigationTarget.AttendanceCapture(payload.direction, payload.store)
        "enroll"   -> NavigationTarget.DeviceEnrollment(payload.enrollmentToken)
        "ship"     -> NavigationTarget.ShipmentDetail(payload.shipmentId, userRole)
        "task"     -> NavigationTarget.TaskDetail(payload.taskId)
        "approval" -> NavigationTarget.ApprovalForm(payload.approvalId)
        else       -> NavigationTarget.UnknownQR(scanResult.data)
    }
}
```

WhatsApp QR codes (receipt, customer_reg, product details) use the `wa.me` URL format and open WhatsApp instead of the Posterita app. These are for customers, not staff.

Internal QR codes (shelf, pkg, attendance, enroll, shipment, task, approval) use JSON payloads and open directly in the Posterita app. These are for staff.

Attendance QR Stations (expanded)

Every store has QR codes printed at the entrance:

```
┌──────────────────────────┐    ┌──────────────────────────┐
│        ┌──────────┐      │    │        ┌──────────┐      │
│        │ QR CODE  │      │    │        │ QR CODE  │      │
│        │  CHECK   │      │    │        │  CHECK   │      │
│        │   IN     │      │    │        │   OUT    │      │
│        └──────────┘      │    │        └──────────┘      │
│                          │    │                          │
│   Scan to clock in       │    │   Scan to clock out      │
│   Grand Baie Store       │    │   Grand Baie Store       │
│   Posterita Retail OS    │    │   Posterita Retail OS    │
└──────────────────────────┘    └──────────────────────────┘
```

Staff scans → app opens → attendance captured instantly (auto-submit with GPS + timestamp). No form to fill. Just scan and go.

The QR token includes a hash to prevent forgery: `{"t":"att","dir":"in","store":"GB","hash":"HMAC(secret, GB+in)"}`. Backend validates the hash.

---

33. Operational Supplies (Non-Resale Inventory)

The Problem

Stores need items that aren't sold to customers: receipt rolls, shopping bags, cleaning supplies, uniforms, brooms, bins, stationery, printer paper, etc. These need to be tracked, ordered, and dispatched from the warehouse — but they must NOT mix with the resale product catalogue.

Design: Same Tables, Strict Separation

Operational supplies reuse the product infrastructure (same scanning, same inventory count, same logistics) but are completely separated by a `product_class` field:

```
-- Add to existing product table:
ALTER TABLE product ADD COLUMN product_class TEXT NOT NULL DEFAULT 'resale'
    CHECK (product_class IN ('resale', 'operational'));

-- Cost and pricing (v3.7 — container receiving)
ALTER TABLE product ADD COLUMN selling_price NUMERIC(12,2);       -- set immediately when product created
ALTER TABLE product ADD COLUMN cost_price NUMERIC(12,2);          -- NULL until container costed (sell now, cost later)
ALTER TABLE product ADD COLUMN cost_status TEXT NOT NULL DEFAULT 'pending'
    CHECK (cost_status IN ('pending', 'costed', 'estimated'));    -- 'pending' = not yet calculated from container
ALTER TABLE product ADD COLUMN container_id UUID REFERENCES container(id);  -- which container this product came from
ALTER TABLE product ADD COLUMN supplier_name TEXT;
ALTER TABLE product ADD COLUMN supplier_sku TEXT;                  -- supplier's own product code
```

Enforcement rules: - POS product grid ONLY shows `product_class = 'resale'` — operational supplies never appear in the sales screen - Inventory count can count both classes but reports them separately - Catalogue PDF generation only includes `resale` products - Logistics shipments can contain both classes (a carton might have sandals + receipt rolls) - Warehouse dashboard shows both with clear visual separation - Stock alerts work for both classes independently - AI enrichment only runs on `resale` products (operational supplies don't need marketing copy)

Pre-built Operational Supply Categories

The system ships with a default `operational` category structure:

Category	Example Items
POS Consumables	Receipt rolls, barcode label rolls, printer ribbons, shopping bags
Cleaning	Brooms, mops, bins, cleaning sprays, garbage bags
Uniforms	Staff shirts, aprons, name tags, lanyards
Stationery	Pens, notebooks, cash count sheets, filing folders
Store Fixtures	Hangers, price tag guns, sign holders, shelf dividers
Packaging	Carton boxes, bubble wrap, tape, gift bags
Safety	First aid kit, fire extinguisher, safety signs

Warehouse Dispatch Flow

```
STORE requests operational supplies (stationery, receipt rolls, etc.)
  │
  ▼
Request goes to warehouse queue
  │
  ▼
WAREHOUSE STAFF picks items → creates shipment → prints package labels
  │
  ▼
DRIVER delivers to store (standard parcel template)
  │
  ▼
STORE scans package QR → confirms receipt → stock levels update
```

Reorder Alerts

```
-- Operational supplies can have minimum stock thresholds
ALTER TABLE product ADD COLUMN min_stock_threshold INTEGER;
ALTER TABLE product ADD COLUMN reorder_quantity INTEGER;
```

When an operational supply's stock falls below `min_stock_threshold`, the system: 1. Creates an alert in the web console dashboard 2. Optionally auto-creates a supply request to the warehouse 3. Sends a notification to the store supervisor

What This Doesn't Cover (Yet)

- Purchase orders to external suppliers (the system tracks internal dispatch, not procurement)
- Supplier management
- Cost allocation of operational supplies to stores
- Depreciation of store fixtures

---

34. "Barcode My Store" — Guided First-Time Barcoding

The Problem

Many retail stores — especially small shops and new businesses — have never barcoded their products. Their products sit on shelves with handwritten price tags or no tags at all. Before they can use POS scanning, inventory counting, or any of the Retail OS scanning features, every product needs a barcode.

This is a massive adoption barrier. If barcoding requires manually entering every product into a system, most store owners give up.

The Solution: Shelf-by-Shelf Photo Capture + AI Identification

The "Barcode My Store" feature is a **guided workflow** that turns a chaotic unbarcoded store into a fully barcoded one, shelf by shelf, using only a phone camera.

The Flow

```
BARCODE MY STORE — Setup Wizard
═══════════════════════════════

STEP 1: PREP
┌──────────────────────────────┐
│                              │
│   Let's barcode your store!  │
│                              │
│   Before we start:           │
│   • Group similar items on   │
│     each shelf               │
│   • One product type per     │
│     group (all sizes together)│
│   • Clear shelf labels help  │
│     but aren't required      │
│                              │
│       [ Start → ]            │
│                              │
└──────────────────────────────┘

STEP 2: SCAN SHELF (or create one)
┌──────────────────────────────┐
│                              │
│   Scan shelf QR or enter     │
│   shelf location             │
│                              │
│   [📷 Scan Shelf QR]         │
│                              │
│   — or —                     │
│                              │
│   No shelf labels yet?       │
│   [Create Shelf: Zone __ Shelf __]
│                              │
│   This could also be a carton│
│   box, a display rack, a table│
│                              │
└──────────────────────────────┘

STEP 3: HOW MANY DIFFERENT PRODUCTS?
┌──────────────────────────────┐
│                              │
│   Shelf: GB-001-003A         │
│                              │
│   How many different products│
│   are on this shelf?         │
│                              │
│   (not total items — just    │
│    different product types)  │
│                              │
│   [ 1 ] [ 2 ] [ 3 ] [ 4 ]    │
│   [ 5 ] [ 6 ] [ 7 ] [ 8+ ]   │
│                              │
└──────────────────────────────┘

STEP 4: PHOTOGRAPH EACH PRODUCT (repeat for each)
┌──────────────────────────────┐
│                              │
│   Product 1 of 4             │
│   Shelf: GB-001-003A         │
│                              │
│   📷 Take a photo of this    │
│   product                    │
│                              │
│   ┌────────────────────┐     │
│   │                    │     │
│   │   [camera preview] │     │
```

```
│   │                        │   │
│   └────────────────────────┘   │
│                                │
│   Tip: show the front label    │
│   or brand name clearly        │
│                                │
│        [ 📷 Capture ]          │
│                                │
└────────────────────────────────┘

STEP 5: ENTER QUANTITY FOR EACH
┌────────────────────────────────┐
│                                │
│   Product 1 of 4               │
│   Shelf: GB-001-003A           │
│                                │
│   [photo thumbnail]            │
│                                │
│   How many of this item?       │
│                                │
│   [ 12 ]                       │
│                                │
│   Price (optional now):        │
│   [ Rs ______ ]                │
│                                │
│        [ Next Product → ]      │
│                                │
└────────────────────────────────┘

STEP 6: SHELF COMPLETE — NEXT SHELF?
┌────────────────────────────────┐
│                                │
│   ✓ Shelf GB-001-003A done!    │
│   4 products · 47 items        │
│                                │
│   [Next Shelf →] [Done for now] │
│                                │
│   Progress: 3 of ~12 shelves   │
│   ━━━━━━━●━━━━━━ 25%           │
│                                │
└────────────────────────────────┘
```

After Capture: AI Product Identification

Once photos are captured, the AI enrichment pipeline (§29) processes them:

1. **AI examines each photo** — identifies product name, brand (if visible), category, color, size
2. **AI suggests a product name and description** — based on what it sees in the image
3. **AI groups potential duplicates** — "This looks like the same sandal you photographed on shelf 2"
4. **Owner reviews suggestions one by one** — same Accept/Edit/Skip UI as onboarding product review

```
AI IDENTIFIED PRODUCT
┌────────────────────────────────┐
│                                │
│   Shelf GB-001-003A · Item 1   │
│                                │
│   [photo]                      │
│                                │
│   🤖 AI suggests:              │
│                                │
│   Name: Reef Pro Sandal Navy   │
│   Category: Footwear           │
│   Confidence: 72%              │
│                                │
│   Qty on shelf: 12             │
│   Price: (not set)             │
│                                │
│   [✓ Accept] [✎ Edit] [✗ Skip] │
│                                │
└────────────────────────────────┘

IF EDIT:
┌────────────────────────────────┐
│                                │
│   Name:                        │
│   [Men's Canvas Slip-On Blue ] │
│                                │
│   Category:                    │
│   [Footwear              ▾  ]  │
│                                │
│   Price:                       │
│   [Rs 890                  ]   │
│                                │
│         [ Save → ]             │
│                                │
└────────────────────────────────┘
```

What Happens After Approval

When the owner accepts (or edits) a product:

1. **Product created in master `product` table** with auto-generated SKU
2. **Barcode assigned** — system generates a unique barcode (EAN-13 or internal format)
3. **Product linked to shelf** — the system knows where this product lives
4. **Initial stock set** — quantity from Step 5 becomes the opening stock count

5. **Product image stored** — the photo becomes the product's primary image in Cloudinary

Label Printing Phase

After all shelves are captured and products approved:

```
PRINT BARCODE LABELS
┌──────────────────────────────┐
│                              │
│   Ready to print labels!     │
│                              │
│   47 products need labels    │
│   Organized by shelf location │
│                              │
│   Print order:               │
│   — GB-001-001A (5 products) │
│   — GB-001-001B (3 products) │
│   — GB-001-002A (7 products) │
│   ...                        │
│                              │
│   Labels print in shelf order │
│   so you walk through the    │
│   store and stick them in    │
│   sequence.                  │
│                              │
│   [🖨 Print All Labels]      │
│   [🖨 Print One Shelf]       │
│                              │
└──────────────────────────────┘
```

Label print order: Labels are printed in **shelf sequence order** (zone → shelf number → position → alphabetical by product name within shelf). This means the person sticking labels walks through the store in a natural path and applies them in order. No searching.

Each label includes: - Product barcode (scannable) - Product name - Price - Shelf location (e.g. "GB-001-003A") - Small QR code linking to product details

Schema Additions

```
-- Track barcode-my-store sessions
CREATE TABLE barcode_session (
    id              UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    store_id        UUID NOT NULL REFERENCES store(id),
    started_by      UUID NOT NULL REFERENCES "user"(id),
    status          TEXT NOT NULL DEFAULT 'in_progress'
                    CHECK (status IN ('in_progress', 'photos_complete', 'ai_processing', 'review', 'labels_printed', 'completed')),
    shelves_total   INTEGER NOT NULL DEFAULT 0,
    shelves_done    INTEGER NOT NULL DEFAULT 0,
    products_found  INTEGER NOT NULL DEFAULT 0,
    products_approved INTEGER NOT NULL DEFAULT 0,
    started_at      TIMESTAMPTZ NOT NULL DEFAULT now(),
    completed_at    TIMESTAMPTZ
);

-- Individual product captures before AI processing
CREATE TABLE barcode_capture (
    id              UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    session_id      UUID NOT NULL REFERENCES barcode_session(id),
    shelf_id        UUID NOT NULL REFERENCES shelf(id),
    position_on_shelf INTEGER NOT NULL,            -- 1, 2, 3... within this shelf
    photo_url       TEXT NOT NULL,                  -- Cloudinary URL
    quantity        INTEGER NOT NULL DEFAULT 1,
    price           NUMERIC(12,2),                  -- optional at capture time
    -- AI results
    ai_suggested_name TEXT,
    ai_suggested_category TEXT,
    ai_confidence   NUMERIC(3,2),
    ai_duplicate_of UUID REFERENCES barcode_capture(id),  -- AI thinks this is same as another
    -- Resolution
    status          TEXT NOT NULL DEFAULT 'captured'
                    CHECK (status IN ('captured', 'ai_processed', 'accepted', 'edited', 'skipped', 'duplicate')),
    product_id      UUID REFERENCES product(id),  -- set when accepted → product created
    reviewed_at     TIMESTAMPTZ,
    created_at      TIMESTAMPTZ NOT NULL DEFAULT now()
);
```

API Routes

- `POST /v1/barcode-my-store/sessions` — Start a barcoding session
- `POST /v1/barcode-my-store/sessions/{id}/shelves/{shelf_id}/capture` — Capture product photo on shelf
- `POST /v1/barcode-my-store/sessions/{id}/ai-process` — Trigger AI identification for all captures
- `GET /v1/barcode-my-store/sessions/{id}/review` — Get captures with AI suggestions for review
- `POST /v1/barcode-my-store/captures/{id}/accept` — Accept AI suggestion → create product
- `POST /v1/barcode-my-store/captures/{id}/edit` — Edit and accept → create product
- `POST /v1/barcode-my-store/captures/{id}/skip` — Skip this capture
- `POST /v1/barcode-my-store/captures/{id}/duplicate` — Mark as duplicate of another product
- `POST /v1/barcode-my-store/sessions/{id}/print-labels` — Generate labels in shelf walking order
- `GET /v1/barcode-my-store/sessions/{id}/progress` — Session progress dashboard

Integration with Existing Systems

- **Shelf register:** Uses the same `shelf` table from inventory count. If shelves don't exist yet, the workflow creates them on the fly.
- **Product enrichment:** Accepted products are optionally sent to the AI enrichment pipeline (§29) for richer descriptions. The photo from capture becomes the basis for enrichment.
- **Inventory count:** After barcoding, the store's first "inventory count" is essentially already done — the opening stock quantities from capture become the baseline.
- **POS:** Products appear in the POS grid immediately after acceptance. The store can start scanning and selling right away.

---

35. Container & Import Receiving

The Problem

When a shipping container arrives at the warehouse, it carries goods that need to pass through a gauntlet: document verification, physical inspection, damage assessment, cost allocation, product creation, and finally release to stores. Today this is scattered across spreadsheets, WhatsApp photos, and filing cabinets. Mistakes here mean wrong costs, missing claims, and inventory errors that ripple through the entire operation.

Design Principles for This Module

1. **Document vault first** — every container has a file folder. Everything lives there.
2. **Inspect before you trust** — photograph, verify, document damage before goods enter the system.
3. **Don't block operations** — if one box is inspected and ready, send it to the store. Don't wait for the whole container.
4. **Selling price first, cost later** — products can go on sale before the full landed cost is calculated. Costing catches up.
5. **Total must reconcile** — the sum of all product costs must equal the total container cost (freight + insurance + duties + local charges + product cost). No leakage.

Container Lifecycle

```
CONTAINER ARRIVES AT WAREHOUSE
    │
    ▼
STEP 1: CREATE CONTAINER RECORD
    Merchandiser creates container in system
    → Container ref (e.g. CNTR-2026-015)
    → Supplier / origin
    → Expected contents (from purchase order if available)
    │
    ▼
STEP 2: DOCUMENT VAULT
    Upload all related documents:
    ├── Commercial invoice
    ├── Packing list
    ├── Bill of lading / airway bill
    ├── Import permit / license
    ├── Certificate of origin
    ├── Insurance certificate
    ├── Customs declaration
    ├── Local charges invoice (port fees, handling)
    ├── Freight invoice
    ├── Duty/tax assessment
    └── Any other relevant documents

    Each document: PDF/image upload to Cloudinary
    Tagged by type, searchable, linked to container
    │
    ▼
STEP 3: PHYSICAL INSPECTION (package by package)
    For each package/carton in the container:
    ├── Scan or create package identifier
    ├── Take photos (condition on arrival)
    ├── Open and inspect contents
    ├── Photograph contents
    ├── Note any damage, shortages, or discrepancies
    ├── Mark package status: OK / Damaged / Short / Wrong item
    │
    │   ⚡ FLEXIBLE: Each package can be processed independently
    │   A box that's inspected and OK can be released immediately
    │   while other boxes are still being processed
    │
    ▼
STEP 4: PRODUCT PROCESSING (per inspected package)
    For each product found in the package:
    ├── Is this an existing product? → Link to product record
    ├── Is this a new product? → Create product:
    │   ├── Name, category, photo (from inspection)
    │   ├── Selling price (SET NOW — store needs this to sell)
    │   ├── Cost price: PENDING (will be calculated from container costs)
    │   ├── Optional: send to AI enrichment for description
    │   └── Generate barcode, print label
    ├── Set quantity received
    ├── Assign to destination store(s)
    │
    ▼
STEP 5: RELEASE TO STORE (per package, not per container)
    Inspected + processed package:
    ├── Create shipment (warehouse → store) using logistics module
    ├── Print package labels
    ├── Products become available in destination store's POS
    ├── Stock levels update at destination
    │
    │   ⚡ This happens AS SOON as a package is ready
    │   Don't wait for the full container to be processed
    │
    ▼
STEP 6: COST ALLOCATION (after all packages processed)
    Total container costs:
    ├── Product cost (from supplier invoices)
    ├── Freight cost
    ├── Insurance
    ├── Customs duty
    ├── Local handling charges
    ├── Port fees
    ├── Any other charges
    │
    Allocation method: proportional by product cost, by weight, or by volume
    │
    Landed cost per product = (product cost + allocated share of overheads)
    │
    System verifies: SUM(all product landed costs) = total container cost
    If mismatch → flag for review
    │
    ▼
STEP 7: CLOSE CONTAINER
    All packages inspected ✓
    All products processed ✓
    All products released to stores ✓
    Cost allocation complete ✓
    Total reconciled ✓
    │
    → Container status: CLOSED
    → Purchase order finalized with actual costs
    → All documents archived in vault
```

Claims Workflow

If damage, shortages, or wrong items are found during inspection:

```
DAMAGE / SHORTAGE FOUND
    │
    ├── Photo evidence captured during inspection (Step 3)
    ├── Merchandiser creates a CLAIM:
    │   ├── Claim type: damage / shortage / wrong_item / quality
    │   ├── Claim against: supplier / freight_company / insurance
    │   ├── Affected products + quantities
    │   ├── Photo evidence (linked from inspection)
    │   ├── Document evidence (linked from vault)
    │   ├── Estimated loss amount
    │   └── Claim narrative
    │
    ├── Claim status tracked: draft → submitted → acknowledged → resolved / rejected
    │
    └── Resolution: credit note / replacement / insurance payout / write-off
        → Financial impact recorded against the container
```

Schema

```
-- ═══════════ CONTAINER ═══════════
CREATE TABLE container (
    id                  UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    organization_id     UUID NOT NULL REFERENCES organization(id),
    container_ref       TEXT NOT NULL UNIQUE,        -- "CNTR-2026-015"
    supplier_name       TEXT,
    supplier_ref        TEXT,                        -- supplier's invoice/order number
    origin_country      TEXT,
    shipping_method     TEXT CHECK (shipping_method IN ('sea', 'air', 'road', 'rail', 'courier')),
    container_number    TEXT,                        -- physical container ID (e.g. MSKU1234567)
    arrival_date        DATE,
    status              TEXT NOT NULL DEFAULT 'created'
                        CHECK (status IN ('created', 'documents_uploaded', 'inspecting',
                                          'processing', 'costing', 'closed', 'cancelled')),
    -- Cost summary (populated during costing step)
    total_product_cost  NUMERIC(14,2) DEFAULT 0,
    total_freight       NUMERIC(14,2) DEFAULT 0,
    total_insurance     NUMERIC(14,2) DEFAULT 0,
    total_duty          NUMERIC(14,2) DEFAULT 0,
    total_local_charges NUMERIC(14,2) DEFAULT 0,
    total_other_charges NUMERIC(14,2) DEFAULT 0,
    total_landed_cost   NUMERIC(14,2) DEFAULT 0,     -- sum of all above
    cost_allocation_method TEXT DEFAULT 'proportional_cost'
                        CHECK (cost_allocation_method IN ('proportional_cost', 'proportional_weight', 'proportional_volume', 'manual')),
    cost_reconciled     BOOLEAN NOT NULL DEFAULT false,
    currency            TEXT NOT NULL DEFAULT 'MUR',
    exchange_rate       NUMERIC(12,6),               -- if supplier invoice in foreign currency
    notes               TEXT,
    created_by          UUID NOT NULL REFERENCES "user"(id),
    created_at          TIMESTAMPTZ NOT NULL DEFAULT now(),
    updated_at          TIMESTAMPTZ NOT NULL DEFAULT now(),
    closed_at           TIMESTAMPTZ
);

-- ═══════════ CONTAINER DOCUMENT VAULT ═══════════
CREATE TABLE container_document (
    id              UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    container_id    UUID NOT NULL REFERENCES container(id),
    document_type   TEXT NOT NULL
                    CHECK (document_type IN (
                        'commercial_invoice', 'packing_list', 'bill_of_lading',
                        'airway_bill', 'import_permit', 'certificate_of_origin',
                        'insurance_certificate', 'customs_declaration',
                        'duty_assessment', 'freight_invoice', 'local_charges',
                        'supplier_order', 'quality_certificate', 'license',
                        'permit', 'other'
                    )),
    file_name       TEXT NOT NULL,
    file_url        TEXT NOT NULL,               -- Cloudinary URL
    file_size       INTEGER,
    description     TEXT,
    uploaded_by     UUID NOT NULL REFERENCES "user"(id),
    uploaded_at     TIMESTAMPTZ NOT NULL DEFAULT now()
);

-- ═══════════ CONTAINER PACKAGE (carton/crate within container) ═══════════
CREATE TABLE container_package (
    id              UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    container_id    UUID NOT NULL REFERENCES container(id),
    package_ref     TEXT NOT NULL,               -- "CNTR-2026-015-PKG-01"
    package_type    TEXT DEFAULT 'carton'
                    CHECK (package_type IN ('carton', 'crate', 'pallet', 'loose', 'vehicle', 'other')),
    description     TEXT,
    expected_contents TEXT,                       -- from packing list
    status          TEXT NOT NULL DEFAULT 'pending'
                    CHECK (status IN ('pending', 'inspecting', 'inspected', 'processing',
                                      'ready_to_release', 'released', 'on_hold')),
    condition       TEXT DEFAULT 'not_inspected'
                    CHECK (condition IN ('not_inspected', 'good', 'damaged', 'short', 'wrong_item')),
    inspected_by    UUID REFERENCES "user"(id),
    inspected_at    TIMESTAMPTZ,
    destination_store_id UUID REFERENCES store(id),
    shipment_id     UUID REFERENCES shipment(id), -- linked when released via logistics
    released_at     TIMESTAMPTZ,
    notes           TEXT,
    created_at      TIMESTAMPTZ NOT NULL DEFAULT now()
);

-- ═══════════ CONTAINER INSPECTION PHOTO ═══════════
CREATE TABLE container_inspection_photo (
    id              UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    container_id    UUID NOT NULL REFERENCES container(id),
    package_id      UUID REFERENCES container_package(id),  -- NULL for container-level photos
```

```
    photo_type      TEXT NOT NULL
                    CHECK (photo_type IN ('arrival_exterior', 'arrival_interior', 'package_sealed',
                                          'package_opened', 'contents', 'damage', 'label', 'other')),
    photo_url       TEXT NOT NULL,                -- Cloudinary URL
    description     TEXT,
    is_evidence     BOOLEAN NOT NULL DEFAULT false, -- marked as claim evidence
    captured_by     UUID NOT NULL REFERENCES "user"(id),
    captured_at     TIMESTAMPTZ NOT NULL DEFAULT now()
);

-- ========== CONTAINER PRODUCT LINE (products found in container) ==========
CREATE TABLE container_product_line (
    id              UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    container_id    UUID NOT NULL REFERENCES container(id),
    package_id      UUID NOT NULL REFERENCES container_package(id),
    product_id      UUID REFERENCES product(id),             -- NULL until product created/linked
    description     TEXT NOT NULL,                            -- from packing list or inspection
    quantity_expected INTEGER,                                 -- from packing list
    quantity_received INTEGER NOT NULL,                        -- actually counted
    quantity_damaged  INTEGER NOT NULL DEFAULT 0,
    unit_cost_supplier NUMERIC(12,2),                         -- ex-works cost in supplier currency
    unit_cost_landed   NUMERIC(12,2),                         -- calculated after cost allocation
    selling_price      NUMERIC(12,2),                         -- can be set before cost is calculated
    status          TEXT NOT NULL DEFAULT 'pending'
                    CHECK (status IN ('pending', 'product_created', 'released', 'on_hold')),
    notes           TEXT,
    created_at      TIMESTAMPTZ NOT NULL DEFAULT now()
);

-- ========== CLAIMS ==========
CREATE TABLE container_claim (
    id              UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    container_id    UUID NOT NULL REFERENCES container(id),
    claim_ref       TEXT NOT NULL UNIQUE,         -- "CLM-2026-015-01"
    claim_type      TEXT NOT NULL
                    CHECK (claim_type IN ('damage', 'shortage', 'wrong_item', 'quality', 'other')),
    claim_against   TEXT NOT NULL
                    CHECK (claim_against IN ('supplier', 'freight', 'insurance', 'port', 'other')),
    status          TEXT NOT NULL DEFAULT 'draft'
                    CHECK (status IN ('draft', 'submitted', 'acknowledged', 'under_review',
                                      'resolved', 'rejected', 'escalated')),
    affected_products JSONB,                      -- [{product_line_id, qty, description}]
    estimated_loss  NUMERIC(12,2),
    currency        TEXT NOT NULL DEFAULT 'MUR',
    narrative       TEXT,                         -- detailed description of the issue
    -- Resolution
    resolution_type TEXT
                    CHECK (resolution_type IN ('credit_note', 'replacement', 'insurance_payout',
                                               'write_off', 'partial_credit', NULL)),
    resolution_amount NUMERIC(12,2),
    resolution_notes TEXT,
    resolved_at     TIMESTAMPTZ,
    resolved_by     UUID REFERENCES "user"(id),
    created_by      UUID NOT NULL REFERENCES "user"(id),
    created_at      TIMESTAMPTZ NOT NULL DEFAULT now(),
    updated_at      TIMESTAMPTZ NOT NULL DEFAULT now()
);

-- Link inspection photos to claims as evidence
CREATE TABLE container_claim_evidence (
    id              UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    claim_id        UUID NOT NULL REFERENCES container_claim(id),
    photo_id        UUID REFERENCES container_inspection_photo(id),
    document_id     UUID REFERENCES container_document(id),
    description     TEXT,
    created_at      TIMESTAMPTZ NOT NULL DEFAULT now()
);

-- ========== PURCHASE ORDER (generated from container) ==========
CREATE TABLE purchase_order (
    id              UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    organization_id UUID NOT NULL REFERENCES organization(id),
    container_id    UUID REFERENCES container(id),           -- linked to container if from import
    po_ref          TEXT NOT NULL UNIQUE,                     -- "PO-2026-015"
    supplier_name   TEXT NOT NULL,
    status          TEXT NOT NULL DEFAULT 'draft'
                    CHECK (status IN ('draft', 'confirmed', 'partially_received', 'fully_received',
                                      'costed', 'closed', 'cancelled')),
    currency        TEXT NOT NULL DEFAULT 'MUR',
    subtotal        NUMERIC(14,2) NOT NULL DEFAULT 0,
    freight         NUMERIC(14,2) NOT NULL DEFAULT 0,
    insurance       NUMERIC(14,2) NOT NULL DEFAULT 0,
    duty            NUMERIC(14,2) NOT NULL DEFAULT 0,
    local_charges   NUMERIC(14,2) NOT NULL DEFAULT 0,
    other_charges   NUMERIC(14,2) NOT NULL DEFAULT 0,
    total           NUMERIC(14,2) NOT NULL DEFAULT 0,        -- must equal sum of all line landed costs
    cost_reconciled BOOLEAN NOT NULL DEFAULT false,
    notes           TEXT,
    created_by      UUID NOT NULL REFERENCES "user"(id),
    created_at      TIMESTAMPTZ NOT NULL DEFAULT now(),
    updated_at      TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE TABLE purchase_order_line (
    id              UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    po_id           UUID NOT NULL REFERENCES purchase_order(id),
    product_id      UUID REFERENCES product(id),
    container_line_id UUID REFERENCES container_product_line(id),
    description     TEXT NOT NULL,
    quantity        INTEGER NOT NULL,
    unit_cost       NUMERIC(12,2) NOT NULL,                   -- supplier cost
    landed_cost     NUMERIC(12,2),                            -- after overhead allocation
    line_total      NUMERIC(14,2) NOT NULL,                   -- qty × unit_cost
    line_total_landed NUMERIC(14,2),                          -- qty × landed_cost
    created_at      TIMESTAMPTZ NOT NULL DEFAULT now()
);
```

Cost Allocation — How Landed Cost Works

The total cost of a container is not just the product prices. It includes freight, insurance, duty, port handling, etc. These overhead costs must be allocated to each product so the business knows the true cost of every item.

```
CONTAINER CNTR-2026-015
    Product costs (from supplier invoices):     Rs 850,000
    Freight:                                     Rs  45,000
    Insurance:                                   Rs  12,000
    Customs duty (15%):                          Rs 127,500
    Local port charges:                          Rs  18,000
    Clearing agent fee:                          Rs   8,500
    ──────────────────────────────────────────────────────
    TOTAL CONTAINER LANDED COST:                Rs 1,061,000

ALLOCATION (proportional to product cost):
    Product A: supplier cost Rs 200,000 → share = 200K/850K = 23.5%
        → allocated overhead = 23.5% × Rs 211,000 = Rs 49,635
        → landed cost = Rs 200,000 + Rs 49,635 = Rs 249,635
        → 500 units → unit landed cost = Rs 499.27

    Product B: supplier cost Rs 50,000 → share = 5.9%
        → allocated overhead = 5.9% × Rs 211,000 = Rs 12,409
        → unit landed cost = (50,000 + 12,409) / 200 = Rs 312.05

VERIFICATION:
    SUM(all product landed costs) MUST = Rs 1,061,000
    If not → discrepancy flagged → merchandiser reviews
```

The "Sell Now, Cost Later" Principle

```
DAY 1: Container arrives
    → Merchandiser inspects Box 1
    → Finds 200 sandals (new product)
    → Creates product: "Beach Sandal Classic"
    → Sets SELLING PRICE: Rs 890 (based on market / brand guidelines)
    → Cost price: PENDING (container not fully costed yet)
    → Releases to Grand Baie store → products appear in POS
    → Store can SELL immediately at Rs 890

DAY 3: Full container processed + costed
    → Landed cost calculated: Rs 312 per unit
    → Cost price updated on product record
    → Margin: Rs 890 - Rs 312 = Rs 578 (65%)
    → All orders from Day 1-3 now have correct COGS for reporting
```

The product table already has both `cost_price` and `selling_price` fields. The selling price is set during inspection (Step 4). The cost price is NULL until cost allocation (Step 6) completes. Reports that need margin data use a flag to show "cost pending" for products without allocated costs.

API Routes

- `POST /v1/containers` — Create container record
- `GET /v1/containers` — List containers (filtered by status, date, supplier)
- `GET /v1/containers/{id}` — Container detail with packages, products, documents, costs
- `PUT /v1/containers/{id}` — Update container details/costs
- `POST /v1/containers/{id}/documents` — Upload document to vault
- `GET /v1/containers/{id}/documents` — List all documents
- `POST /v1/containers/{id}/packages` — Create package within container
- `POST /v1/containers/{id}/packages/{pkg_id}/inspect` — Record inspection (condition, photos)
- `POST /v1/containers/{id}/packages/{pkg_id}/products` — Add product line to package
- `POST /v1/containers/{id}/packages/{pkg_id}/release` — Release package to store (creates shipment)
- `POST /v1/containers/{id}/products/{line_id}/create-product` — Create product from container line (selling price now, cost later)
- `POST /v1/containers/{id}/allocate-costs` — Run cost allocation across all product lines
- `POST /v1/containers/{id}/close` — Close container (requires all reconciled)
- `POST /v1/containers/{id}/claims` — Create claim
- `PUT /v1/containers/{id}/claims/{claim_id}` — Update claim status/resolution
- `POST /v1/containers/{id}/purchase-order` — Generate PO from container data
- `GET /v1/purchase-orders` — List purchase orders
- `GET /v1/purchase-orders/{id}` — PO detail with lines and costs

Android Module

```
:feature:warehouse            ← Container receiving, inspection, document vault, cost allocation
                                Download trigger: tap Warehouse tile (merchandiser role)
```

Added to the on-demand feature module list. Only downloaded by merchandisers.

Web Console Section

Section	Functions
Containers	Container list, detail, document vault, inspection photos, cost entry, allocation, reconciliation, close
Purchase Orders	PO list, detail, line editing, cost reconciliation
Claims	Claim management, evidence linking, status tracking, resolution

Reports (added to §27)

- **Container Receiving Summary** — containers by status, average processing time, claims rate
- **Landed Cost Analysis** — cost breakdown by container, product, overhead category
- **Claim Aging** — open claims by age, type, value
- **Margin Analysis** — selling price vs landed cost per product, per container

---

36. Loyalty API Contract (from Flask code review)

The Flask loyalty API (`routes_award.py`, `routes_balance.py`, `routes_consent.py`, `routes_voucher.py`) defines the POS↔loyalty integration contract. The unified backend MUST preserve these behaviors:

Award Points (`POST /api/v1/award`)

- Input: `phone`, `orderUuid`, `orderTotal`, `currency`, `storeId`, `terminalId`
- **Idempotent on `orderUuid`** — duplicate awards return the original result, never double-award
- Idempotency key format: `POS::{orderUuid}`
- Points calculated: `floor(orderTotal / points_per_currency_unit)`, minimum 1 point per transaction
- Creates: claim record + transaction ledger entry + updates wallet balance
- Auto-creates wallet if customer doesn't have one yet

Check Balance (`GET /api/v1/balance/{phone}`)

- Returns: `points`, `tier` (future), `activeVouchers[]`
- Each voucher includes: `voucherId`, `code`, `discountType` (FIXED/PERCENTAGE), `discountValue`, `expiryDate`, `isUsed`
- Filters out expired vouchers automatically
- Phone normalization: strip spaces/dashes, ensure starts with `+`

Record Consent (`POST /api/v1/consent`)

- Input: `phone`, `consentGranted`, `consentSource`, `brandName`, `storeId`, `terminalId`, `userId`, `consentTimestamp`
- **Consent is independent of wallet** — wallet is created even if consent is denied
- Dual-scope: writes to both `Consent_Master` (channel-level) and `Consent_Brand_Map` (brand-level, with promo/news split)
- Consent source tracked: "POS", "WhatsApp Bot", etc.

Validate Voucher (`POST /api/v1/voucher/validate`)

- Input: `code`, `phone`
- Returns: `valid` (boolean), `discountType`, `discountValue`, `message`
- Checks: voucher exists, belongs to customer, status is "issued", not expired

Redeem Voucher (`POST /api/v1/voucher/redeem`)

- Input: `code`, `phone`, `order_uuid`
- **Atomic:** validate + mark redeemed + log redemption in one transaction
- Returns: `redeemed`, `discountApplied`
- Creates redemption log entry with order reference

Phone Normalization (used across all routes)

```
Strip: spaces, dashes, parentheses
If not starting with "+", prepend "+"
Result: E.164 format, e.g. "+23054239978"
```

All of these behaviors are ported to the NestJS `loyalty` module. The API surface changes slightly (unified under `/v1/loyalty/*` prefix) but the contract semantics are preserved exactly.

---

37. Concerns Resolution Log

All 14 concerns from the initial review have been resolved:

#	Concern	Resolution	Plan Impact
C1	POS codebase status	**Early-stage playground** — no production data, no historical data to preserve. Evaluate for reuse, architecture wins over preservation.	Removed `:feature:pos-legacy`. Build `:feature:pos` fresh. No data migration.
C2	Scanner hardware	**Both camera + Bluetooth HID.** Configurable modes: auto-scan, scan-confirm, continuous batch.	`:core:scanner` designed for both. Scan modes configurable per feature context.
C3	Product catalogue	**Reuse POS product grid.** Same screen, different output mode (PDF catalogue, barcode labels, shelf labels).	Added `:feature:catalogue` module. POS screen is dual-purpose.
C4	Customer volume	**~10,000/year.** WhatsApp costs ~$50-100/mo at this scale.	No budget concern. Standard Meta pricing applies.
C5	Spot check	**Yes in MVP.** Single-scan, reporting/KPI only. Generates Shelf Accuracy KPI to trigger full counts.	Added spot check mode to `inventory_count_session` (session_type field). Spot check report added.
C6	Barcode formats	**Multiple supported:** QR Code (default), Code 128, Code 39, DataMatrix. Configurable per store.	`:core:scanner` decodes all. `:core:printer` generates all.
C7	Label printers	**Zebra (enterprise) + Epson (receipt) + generic (PDF).** `PrinterService` abstracts all.	Added Zebra Link-OS/ZPL support to `:core:printer`.
C8	Points rate	**Configurable per brand.** `organization_loyalty_config` table with `points_per_currency_unit`.	Added config table to schema. Award logic reads config.
C9	Web console scope	Not explicitly answered — **recommendation: phased** (see §13).	Phase 2-3 MVP set identified.
C10	Play Store listing	**Unknown** — Fred will determine.	Remains a Phase 1 blocker.
C11	Blink SDK version	**Unknown** — needs investigation from playground codebase.	Remains a Phase 1 item. Check playground code.
C12	WhatsApp Flows	**Not applied for yet.**	Build button-based fallback first. Apply to Meta in Phase 1.
C13	Flask API routes	**Uploaded and reviewed.** Full contract documented in §28.	Loyalty module preserves exact award/balance/consent/voucher semantics.
C14	Android POS code	**Playground exists.** Not yet uploaded to this conversation.	Will evaluate when uploaded. Plan accounts for both reuse and rebuild paths.

---

38. Remaining Open Items

#	Item	Needed By	Status
1	Play Store listing details (package name, signing key)	Phase 1 start	Unknown — Fred to determine
2	Blink SDK version (check playground codebase)	Phase 1 start	Can be determined from existing playground code
3	Apply for Meta WhatsApp Flows access	Phase 1	Not yet started — button fallback is ready
4	Submit 7 WhatsApp template messages for Meta approval	Phase 2	Templates defined in §25, need submission
5	Upload Android POS playground code for evaluation	Anytime	Fred to upload zip — will inform reuse decisions
6	PITR upgrade trigger threshold	Post-launch	Low priority

This is a living document. Update as decisions are made and implementation reveals new constraints. *v3.7 — 19 March 2026 — 38 sections — Container/Import receiving + Merchandiser role